

BreastCancer

September 16, 2026

```
[1]: import pandas as pd
      from sklearn.datasets import load_breast_cancer
      import seaborn as sns
```

```
[2]: df = load_breast_cancer(as_frame = True).frame
      df
```

```
[2]:
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	\
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1203.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	
..	...	...	...	...	...	
564	21.56	22.39	142.00	1479.0	0.11100	
565	20.13	28.25	131.20	1261.0	0.09780	
566	16.60	28.08	108.30	858.1	0.08455	
567	20.60	29.33	140.10	1265.0	0.11780	
568	7.76	24.54	47.92	181.0	0.05263	

	mean compactness	mean concavity	mean concave points	mean symmetry	\
0	0.27760	0.30010	0.14710	0.2419	
1	0.07864	0.08690	0.07017	0.1812	
2	0.15990	0.19740	0.12790	0.2069	
3	0.28390	0.24140	0.10520	0.2597	
4	0.13280	0.19800	0.10430	0.1809	
..	...	...	...	...	
564	0.11590	0.24390	0.13890	0.1726	
565	0.10340	0.14400	0.09791	0.1752	
566	0.10230	0.09251	0.05302	0.1590	
567	0.27700	0.35140	0.15200	0.2397	
568	0.04362	0.00000	0.00000	0.1587	

	mean fractal dimension	...	worst texture	worst perimeter	worst area	\
0	0.07871	...	17.33	184.60	2019.0	
1	0.05667	...	23.41	158.80	1956.0	
2	0.05999	...	25.53	152.50	1709.0	
3	0.09744	...	26.50	98.87	567.7	

4	0.05883	...	16.67	152.20	1575.0
..	...	...	...	...	...
564	0.05623	...	26.40	166.10	2027.0
565	0.05533	...	38.25	155.00	1731.0
566	0.05648	...	34.12	126.70	1124.0
567	0.07016	...	39.42	184.60	1821.0
568	0.05884	...	30.37	59.16	268.6

	worst smoothness	worst compactness	worst concavity \
0	0.16220	0.66560	0.7119
1	0.12380	0.18660	0.2416
2	0.14440	0.42450	0.4504
3	0.20980	0.86630	0.6869
4	0.13740	0.20500	0.4000
..	...	...	...
564	0.14100	0.21130	0.4107
565	0.11660	0.19220	0.3215
566	0.11390	0.30940	0.3403
567	0.16500	0.86810	0.9387
568	0.08996	0.06444	0.0000

	worst concave points	worst symmetry	worst fractal dimension	target
0	0.2654	0.4601	0.11890	0
1	0.1860	0.2750	0.08902	0
2	0.2430	0.3613	0.08758	0
3	0.2575	0.6638	0.17300	0
4	0.1625	0.2364	0.07678	0
..	...	...	...	...
564	0.2216	0.2060	0.07115	0
565	0.1628	0.2572	0.06637	0
566	0.1418	0.2218	0.07820	0
567	0.2650	0.4087	0.12400	0
568	0.0000	0.2871	0.07039	1

[569 rows x 31 columns]

```
[3]: df.shape # shows the shape of the dataframe
```

```
[3]: (569, 31)
```

```
[4]: df.isnull().sum() # no null records found in the columns shown
```

```
[4]: mean radius          0
     mean texture         0
     mean perimeter       0
     mean area            0
     mean smoothness      0
```

```

mean compactness      0
mean concavity        0
mean concave points   0
mean symmetry         0
mean fractal dimension 0
radius error          0
texture error         0
perimeter error       0
area error            0
smoothness error      0
compactness error     0
concavity error       0
concave points error  0
symmetry error        0
fractal dimension error 0
worst radius          0
worst texture         0
worst perimeter       0
worst area            0
worst smoothness     0
worst compactness     0
worst concavity       0
worst concave points  0
worst symmetry        0
worst fractal dimension 0
target               0
dtype: int64

```

```
[5]: df.info() # shows the general information about the dataset
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   mean radius                          569 non-null    float64
1   mean texture                         569 non-null    float64
2   mean perimeter                      569 non-null    float64
3   mean area                           569 non-null    float64
4   mean smoothness                     569 non-null    float64
5   mean compactness                    569 non-null    float64
6   mean concavity                      569 non-null    float64
7   mean concave points                 569 non-null    float64
8   mean symmetry                       569 non-null    float64
9   mean fractal dimension               569 non-null    float64
10  radius error                        569 non-null    float64
11  texture error                       569 non-null    float64
12  perimeter error                     569 non-null    float64

```

```

13 area error          569 non-null    float64
14 smoothness error    569 non-null    float64
15 compactness error   569 non-null    float64
16 concavity error     569 non-null    float64
17 concave points error 569 non-null    float64
18 symmetry error      569 non-null    float64
19 fractal dimension error 569 non-null    float64
20 worst radius        569 non-null    float64
21 worst texture       569 non-null    float64
22 worst perimeter     569 non-null    float64
23 worst area          569 non-null    float64
24 worst smoothness    569 non-null    float64
25 worst compactness   569 non-null    float64
26 worst concavity     569 non-null    float64
27 worst concave points 569 non-null    float64
28 worst symmetry      569 non-null    float64
29 worst fractal dimension 569 non-null    float64
30 target             569 non-null    int64
dtypes: float64(30), int64(1)
memory usage: 137.9 KB

```

```
[6]: df.drop_duplicates() # dropping the duplicates in the dataframe
```

```

[6]:      mean radius  mean texture  mean perimeter  mean area  mean smoothness \
0          17.99         10.38         122.80      1001.0         0.11840
1          20.57         17.77         132.90      1326.0         0.08474
2          19.69         21.25         130.00      1203.0         0.10960
3          11.42         20.38          77.58       386.1         0.14250
4          20.29         14.34         135.10      1297.0         0.10030
..          ...          ...          ...          ...          ...
564         21.56         22.39         142.00      1479.0         0.11100
565         20.13         28.25         131.20      1261.0         0.09780
566         16.60         28.08         108.30       858.1         0.08455
567         20.60         29.33         140.10      1265.0         0.11780
568          7.76         24.54          47.92       181.0         0.05263

      mean compactness  mean concavity  mean concave points  mean symmetry \
0          0.27760         0.30010         0.14710         0.2419
1          0.07864         0.08690         0.07017         0.1812
2          0.15990         0.19740         0.12790         0.2069
3          0.28390         0.24140         0.10520         0.2597
4          0.13280         0.19800         0.10430         0.1809
..          ...          ...          ...          ...
564         0.11590         0.24390         0.13890         0.1726
565         0.10340         0.14400         0.09791         0.1752
566         0.10230         0.09251         0.05302         0.1590
567         0.27700         0.35140         0.15200         0.2397

```

568	0.04362	0.00000	0.00000	0.1587
-----	---------	---------	---------	--------

	mean fractal dimension	...	worst texture	worst perimeter	worst area	\
0	0.07871	...	17.33	184.60	2019.0	
1	0.05667	...	23.41	158.80	1956.0	
2	0.05999	...	25.53	152.50	1709.0	
3	0.09744	...	26.50	98.87	567.7	
4	0.05883	...	16.67	152.20	1575.0	
..	...	...	...	...	...	
564	0.05623	...	26.40	166.10	2027.0	
565	0.05533	...	38.25	155.00	1731.0	
566	0.05648	...	34.12	126.70	1124.0	
567	0.07016	...	39.42	184.60	1821.0	
568	0.05884	...	30.37	59.16	268.6	

	worst smoothness	worst compactness	worst concavity	\
0	0.16220	0.66560	0.7119	
1	0.12380	0.18660	0.2416	
2	0.14440	0.42450	0.4504	
3	0.20980	0.86630	0.6869	
4	0.13740	0.20500	0.4000	
..	...	...	...	
564	0.14100	0.21130	0.4107	
565	0.11660	0.19220	0.3215	
566	0.11390	0.30940	0.3403	
567	0.16500	0.86810	0.9387	
568	0.08996	0.06444	0.0000	

	worst concave points	worst symmetry	worst fractal dimension	target
0	0.2654	0.4601	0.11890	0
1	0.1860	0.2750	0.08902	0
2	0.2430	0.3613	0.08758	0
3	0.2575	0.6638	0.17300	0
4	0.1625	0.2364	0.07678	0
..	...	...	...	...
564	0.2216	0.2060	0.07115	0
565	0.1628	0.2572	0.06637	0
566	0.1418	0.2218	0.07820	0
567	0.2650	0.4087	0.12400	0
568	0.0000	0.2871	0.07039	1

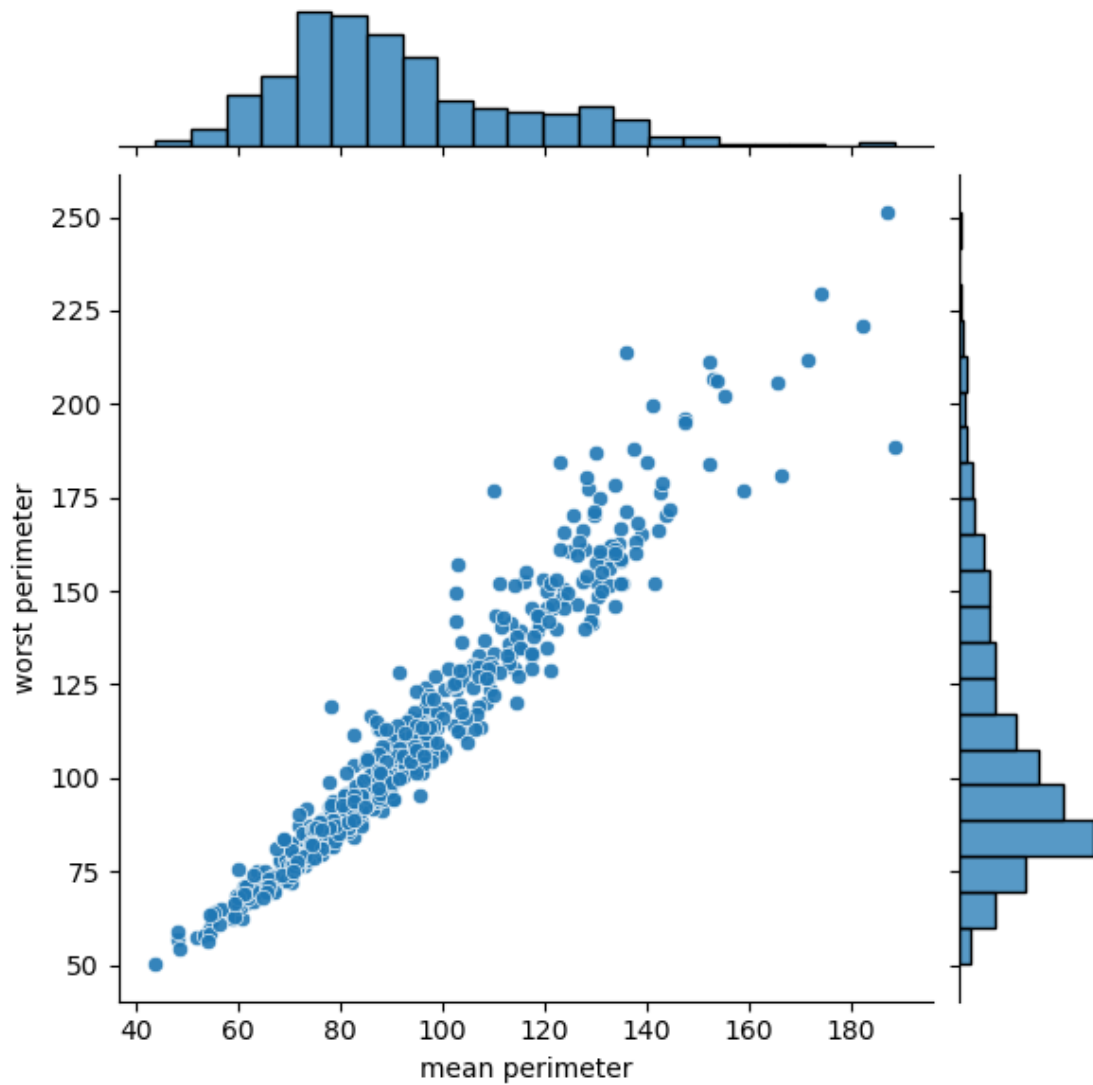
[569 rows x 31 columns]

```
[7]: df["target"].nunique() # shows the total unique values in the target column
```

```
[7]: 2
```

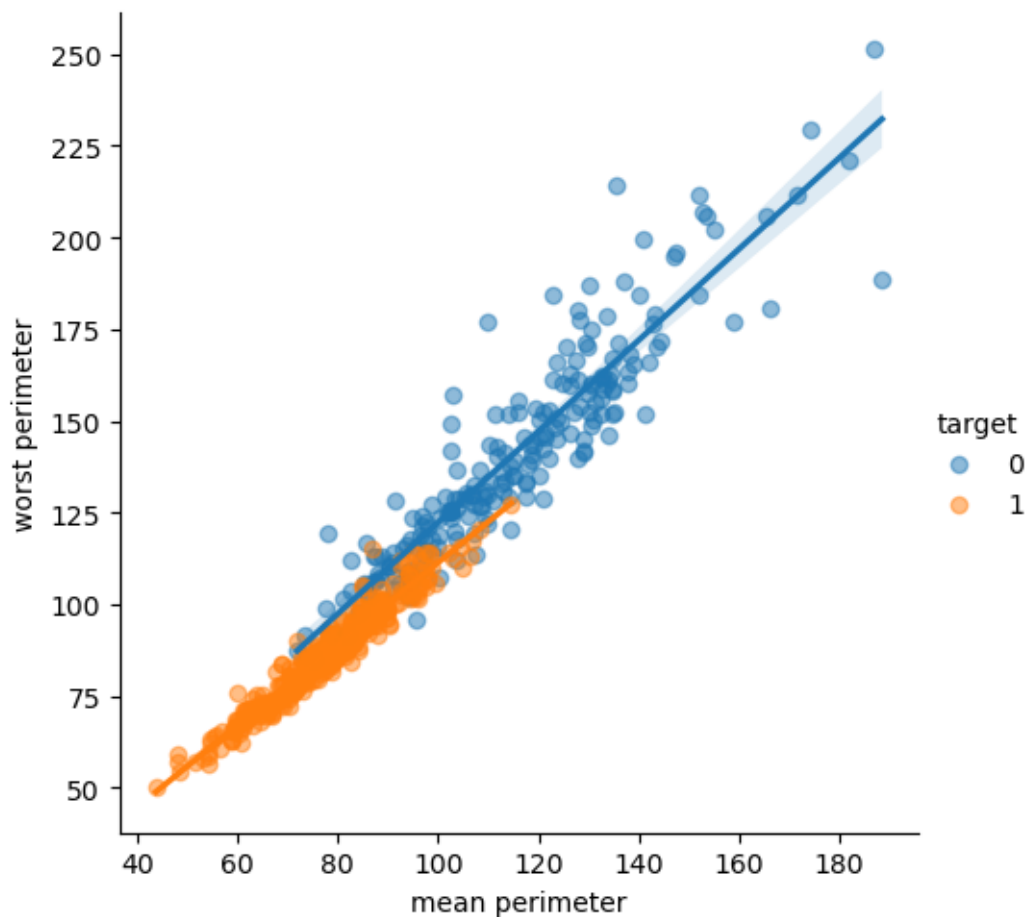
```
[8]: # EDA Explanatory Data Analysis using jointplot
sns.jointplot(x = "mean perimeter",
              y = "worst perimeter", data = df, alpha = 0.9)
```

```
[8]: <seaborn.axisgrid.JointGrid at 0x1936141eba0>
```



```
[9]: sns.lmplot (x = "mean perimeter", # show the X value
                 y = "worst perimeter", # shows the y value
                 scatter_kws = {"alpha" : 0.5 }, # plots the scatter plot
                 data = df, hue = "target")
```

```
[9]: <seaborn.axisgrid.FacetGrid at 0x193616eecf0>
```



```
[10]: df.describe()
```

```
[10]:
```

	mean radius	mean texture	mean perimeter	mean area \
count	569.000000	569.000000	569.000000	569.000000
mean	14.127292	19.289649	91.969033	654.889104
std	3.524049	4.301036	24.298981	351.914129
min	6.981000	9.710000	43.790000	143.500000
25%	11.700000	16.170000	75.170000	420.300000
50%	13.370000	18.840000	86.240000	551.100000
75%	15.780000	21.800000	104.100000	782.700000
max	28.110000	39.280000	188.500000	2501.000000

	mean smoothness	mean compactness	mean concavity	mean concave points \
count	569.000000	569.000000	569.000000	569.000000
mean	0.096360	0.104341	0.088799	0.048919
std	0.014064	0.052813	0.079720	0.038803
min	0.052630	0.019380	0.000000	0.000000
25%	0.086370	0.064920	0.029560	0.020310

50%	0.095870	0.092630	0.061540	0.033500
75%	0.105300	0.130400	0.130700	0.074000
max	0.163400	0.345400	0.426800	0.201200

	mean symmetry	mean fractal dimension	...	worst texture	\
count	569.000000	569.000000	...	569.000000	
mean	0.181162	0.062798	...	25.677223	
std	0.027414	0.007060	...	6.146258	
min	0.106000	0.049960	...	12.020000	
25%	0.161900	0.057700	...	21.080000	
50%	0.179200	0.061540	...	25.410000	
75%	0.195700	0.066120	...	29.720000	
max	0.304000	0.097440	...	49.540000	

	worst perimeter	worst area	worst smoothness	worst compactness	\
count	569.000000	569.000000	569.000000	569.000000	
mean	107.261213	880.583128	0.132369	0.254265	
std	33.602542	569.356993	0.022832	0.157336	
min	50.410000	185.200000	0.071170	0.027290	
25%	84.110000	515.300000	0.116600	0.147200	
50%	97.660000	686.500000	0.131300	0.211900	
75%	125.400000	1084.000000	0.146000	0.339100	
max	251.200000	4254.000000	0.222600	1.058000	

	worst concavity	worst concave points	worst symmetry	\
count	569.000000	569.000000	569.000000	
mean	0.272188	0.114606	0.290076	
std	0.208624	0.065732	0.061867	
min	0.000000	0.000000	0.156500	
25%	0.114500	0.064930	0.250400	
50%	0.226700	0.099930	0.282200	
75%	0.382900	0.161400	0.317900	
max	1.252000	0.291000	0.663800	

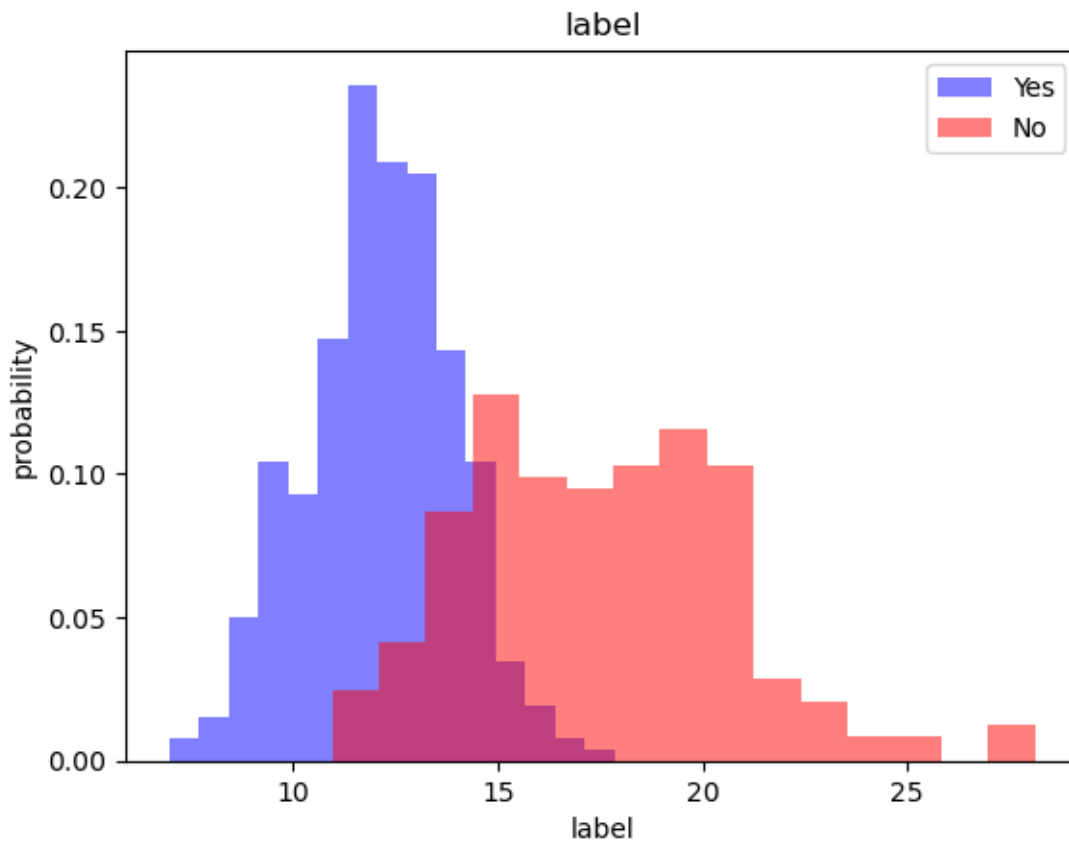
	worst fractal dimension	target
count	569.000000	569.000000
mean	0.083946	0.627417
std	0.018061	0.483918
min	0.055040	0.000000
25%	0.071460	0.000000
50%	0.080040	1.000000
75%	0.092080	1.000000
max	0.207500	1.000000

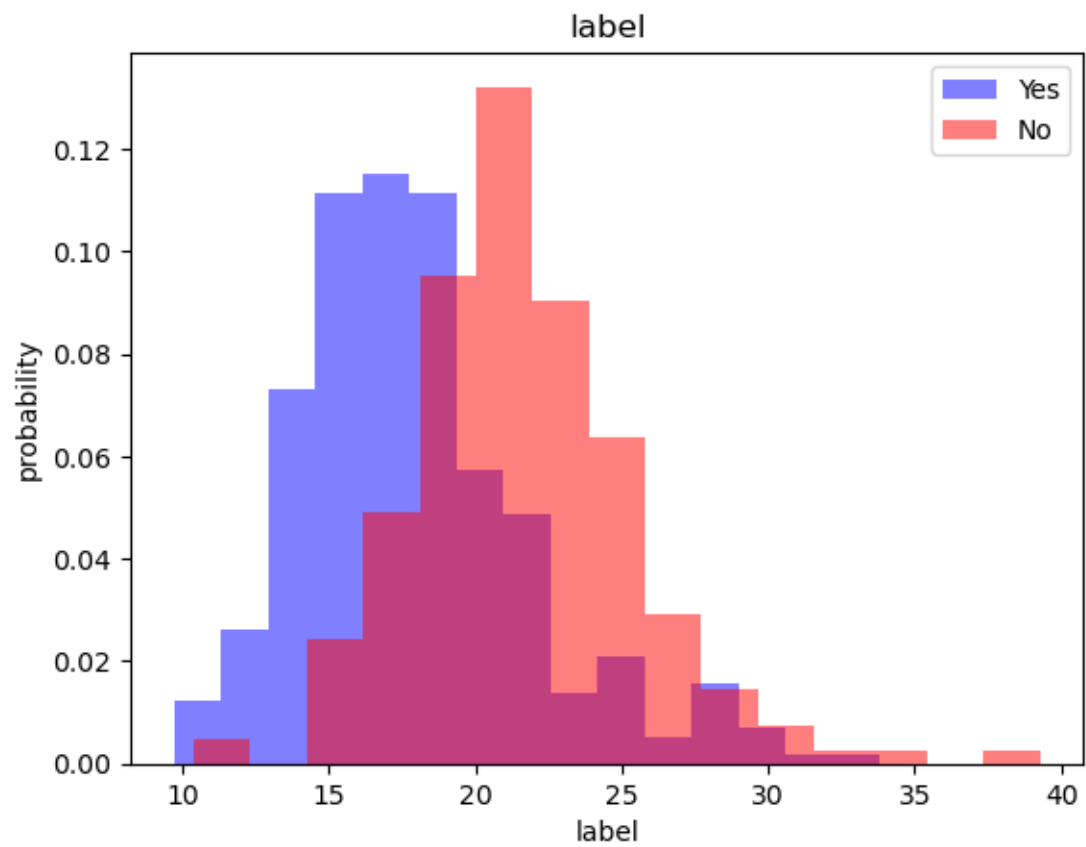
[8 rows x 31 columns]

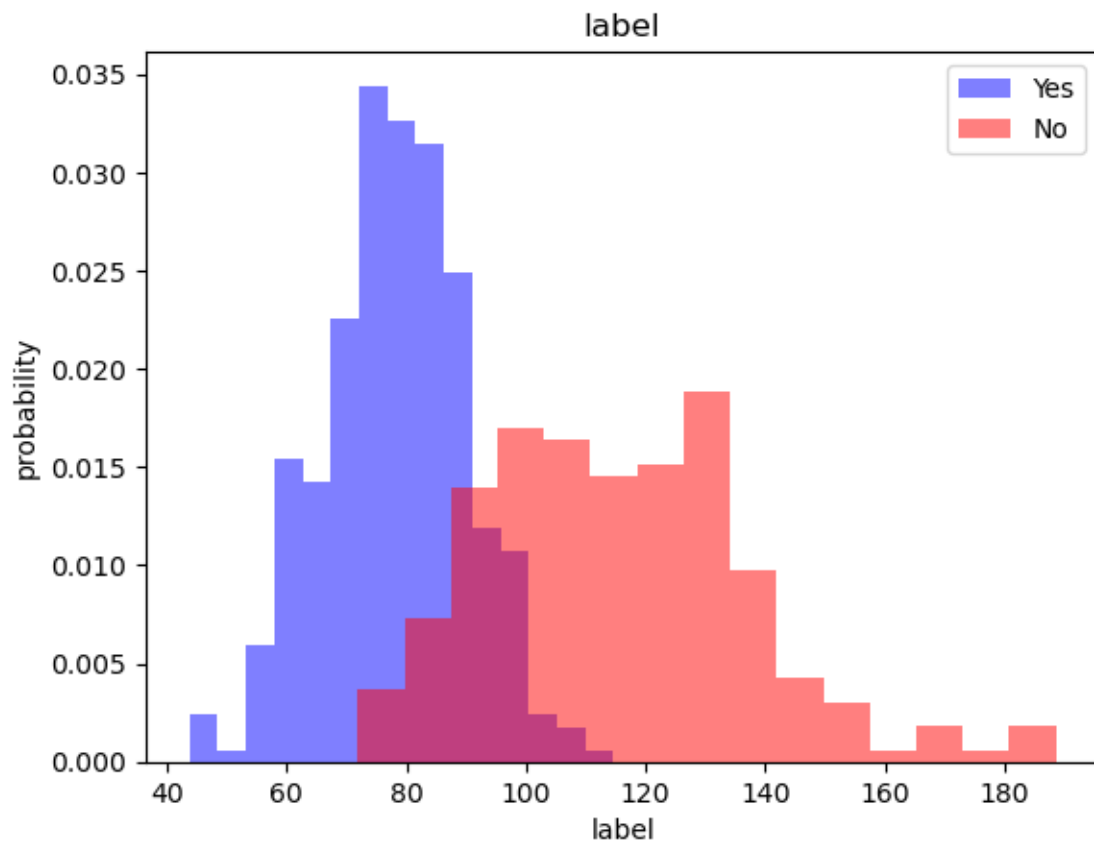
```
[11]: import matplotlib.pyplot as plt
```

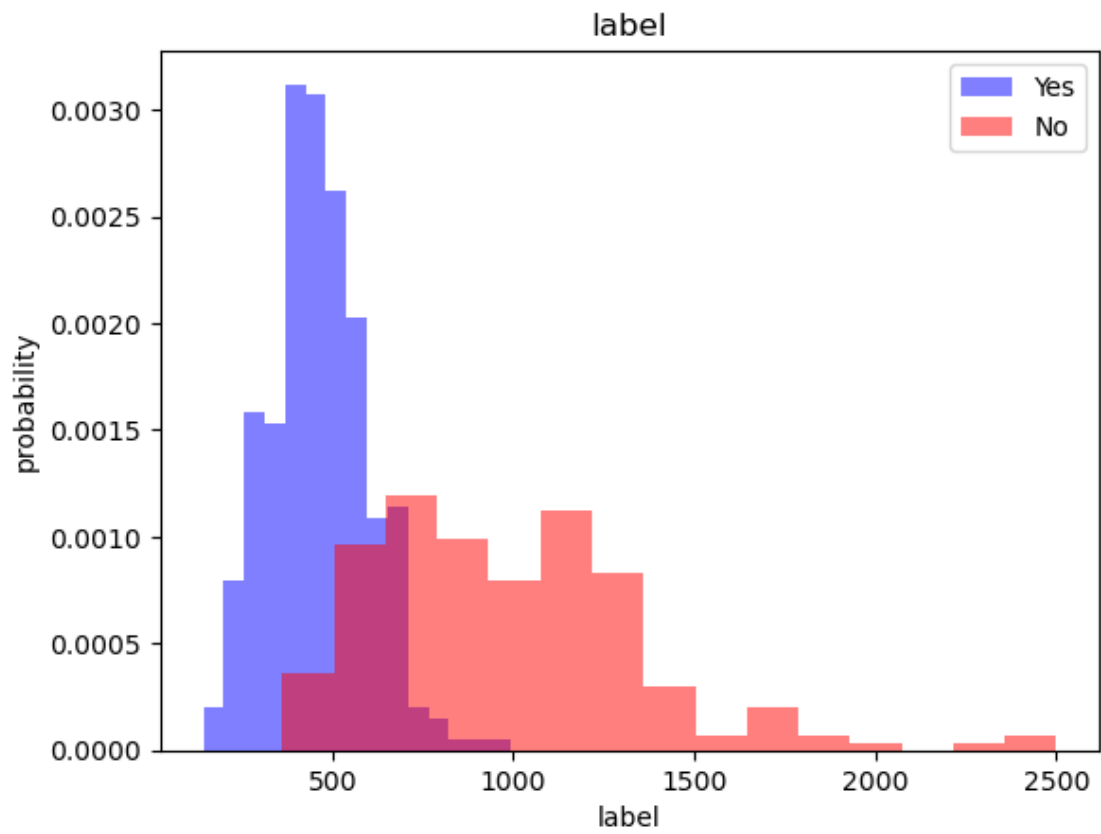
```
[12]: for i in range(len(df.columns[: -1])):
      label = df.columns[i]
      # Line 3
      plt.hist(df[df["target"] == 1][label], color="blue", label="Yes", alpha=0.
      ↪5, density=True, bins=15)

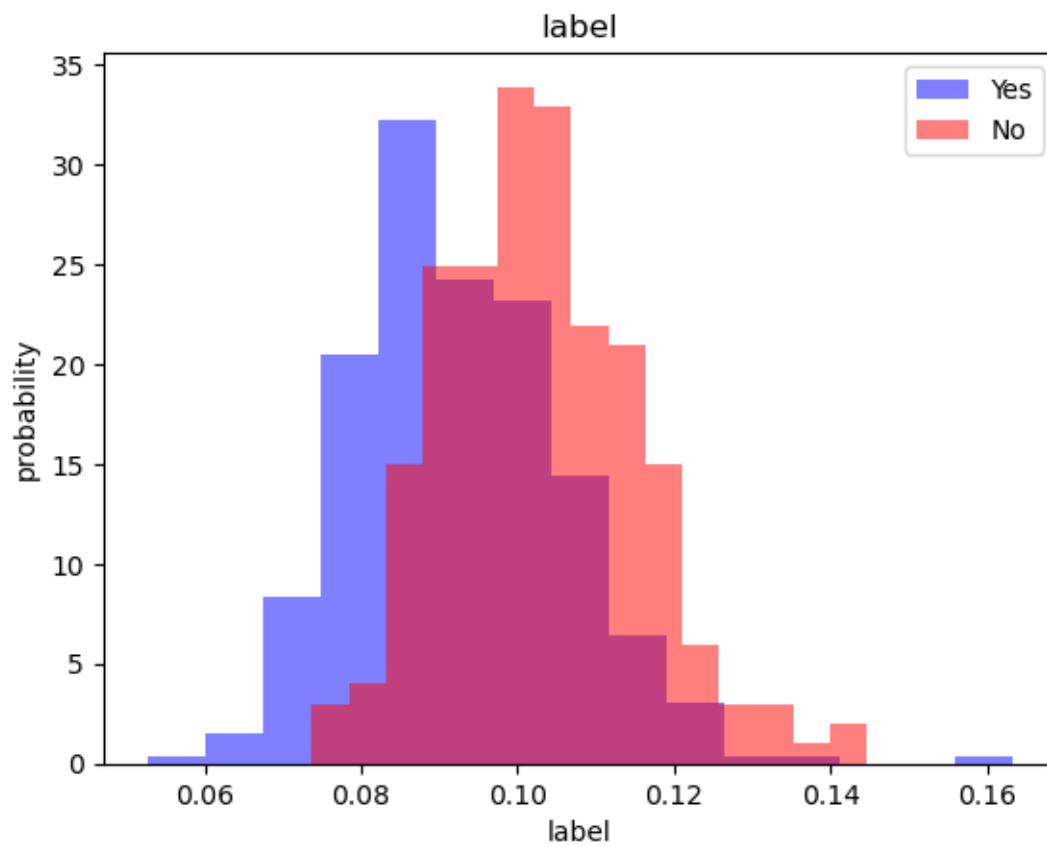
      # Line 4 (This line will likely have the same error)
      plt.hist(df[df["target"] == 0][label], color="red", label="No", alpha=0.5,
      ↪density=True, bins=15)
      plt.title("label")
      plt.xlabel("label")
      plt.ylabel("probability")
      plt.legend()
      plt.show()
```

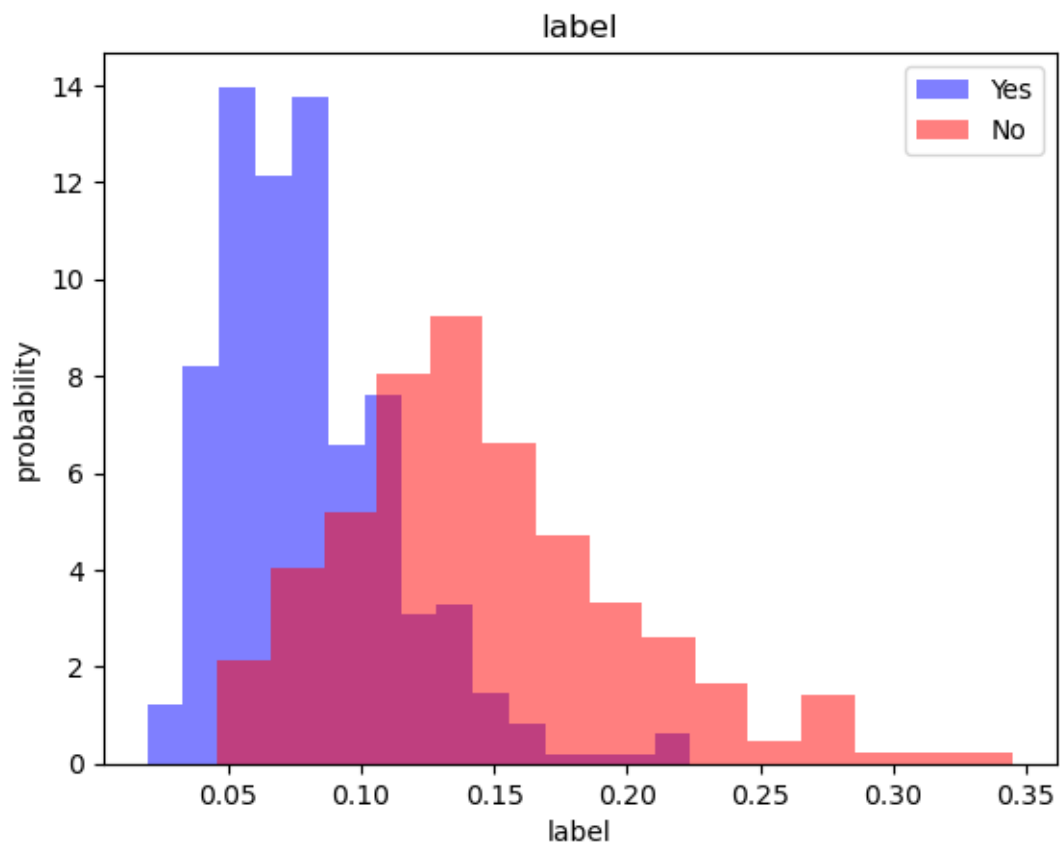


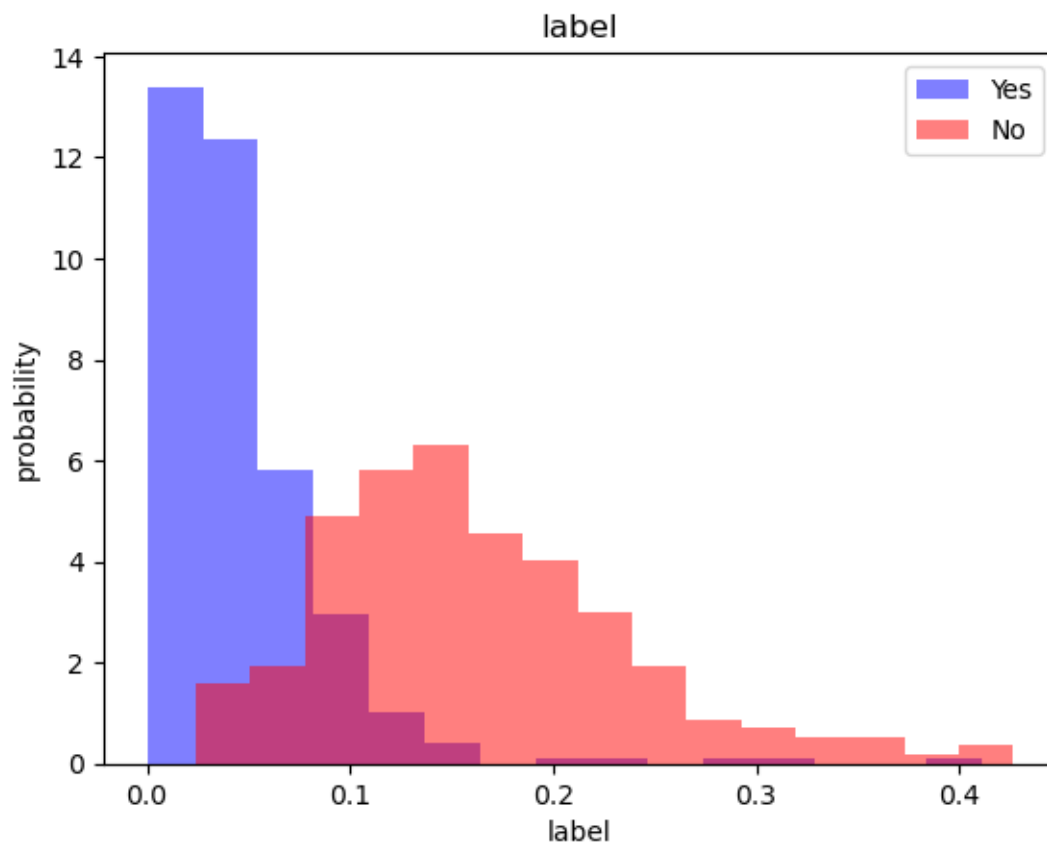


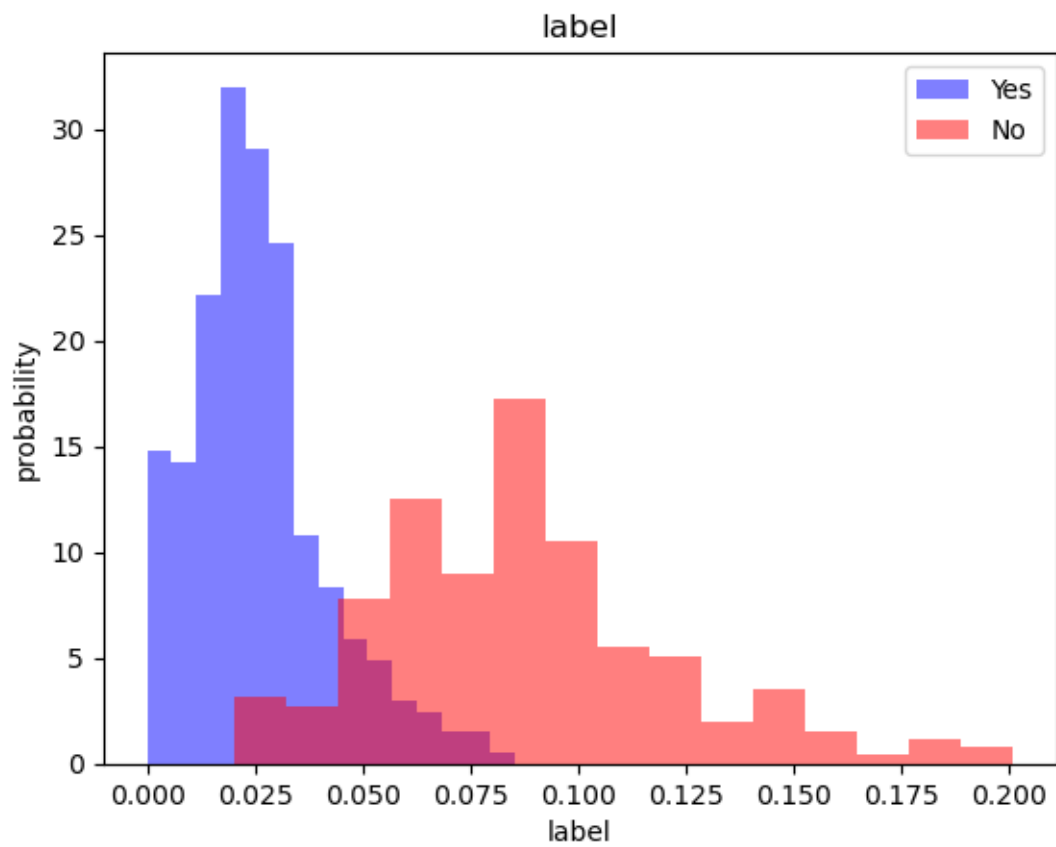


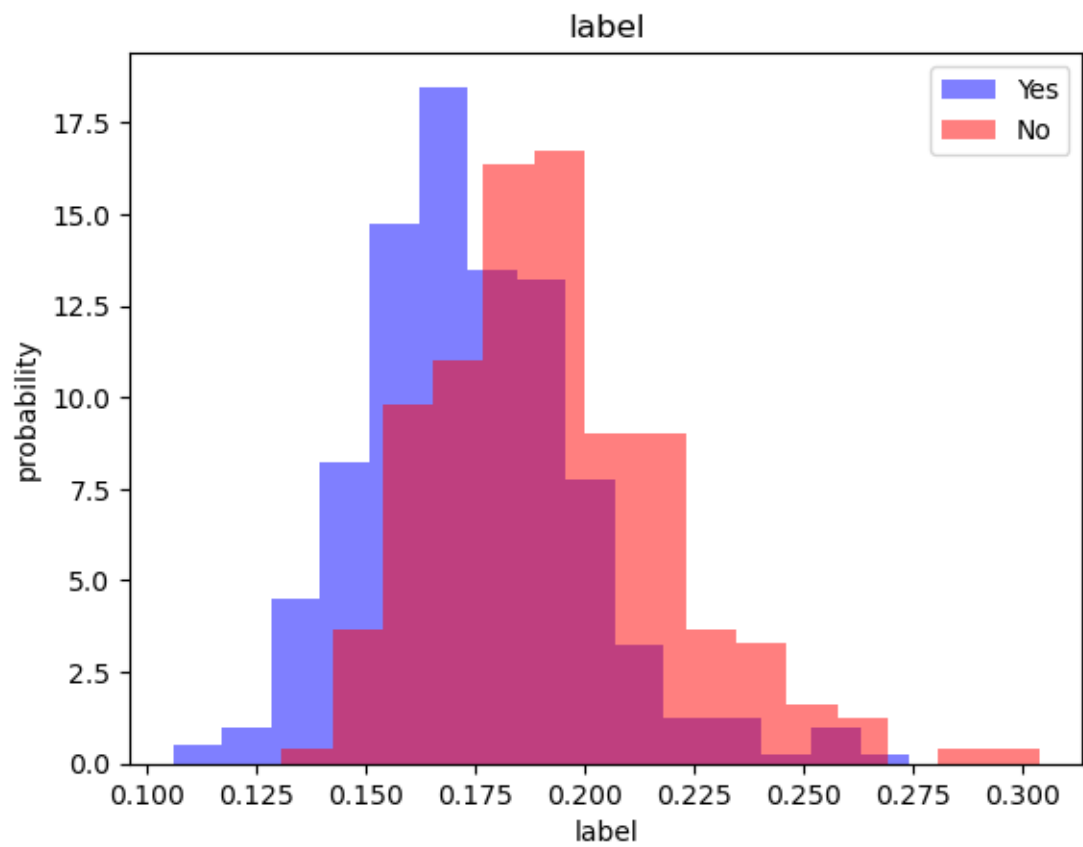


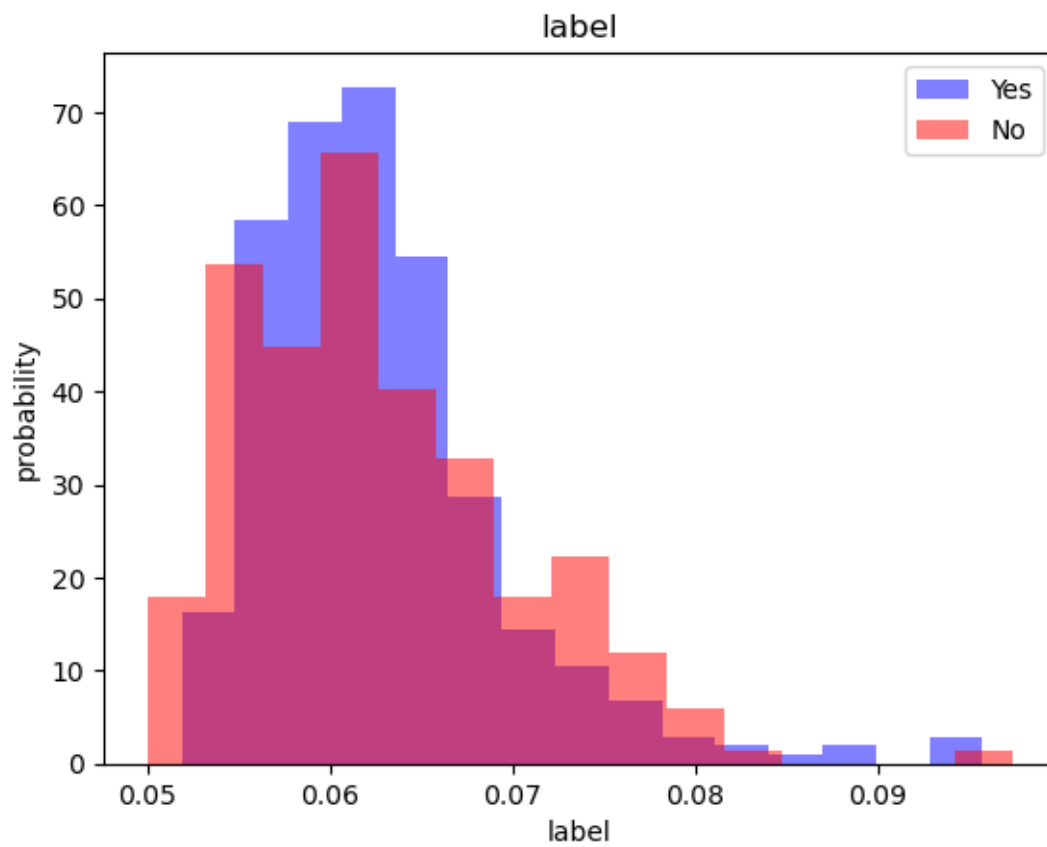


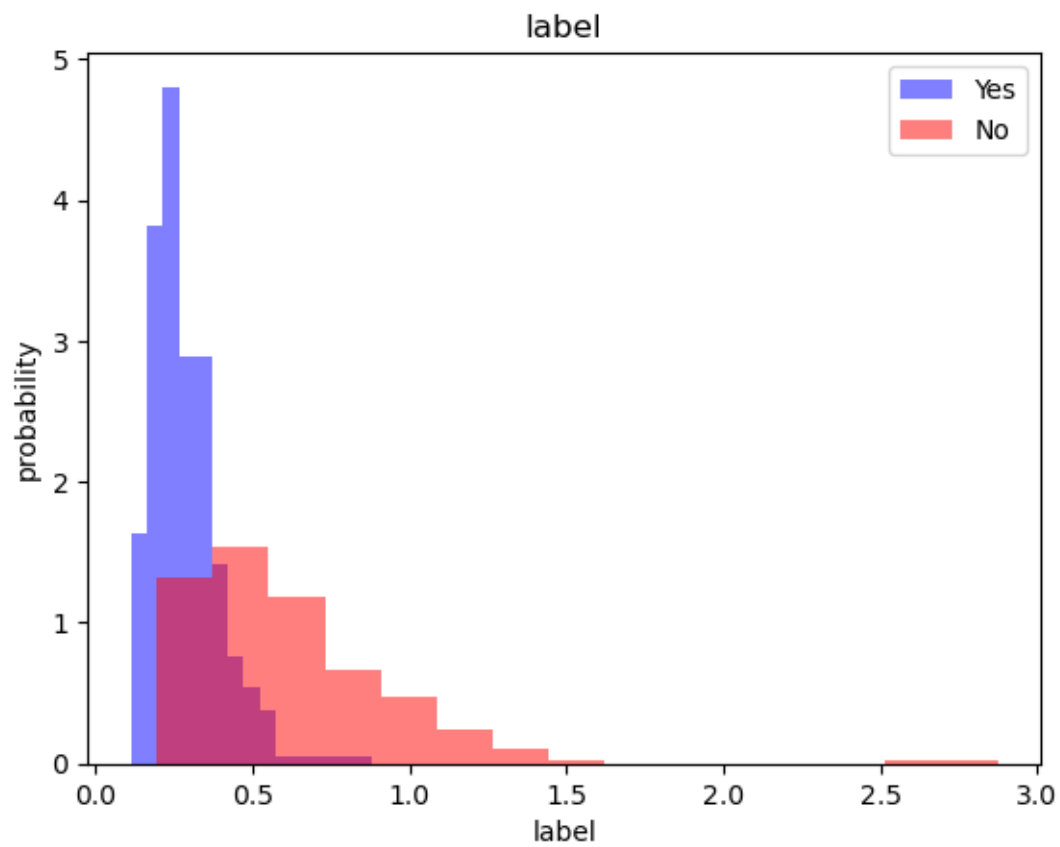


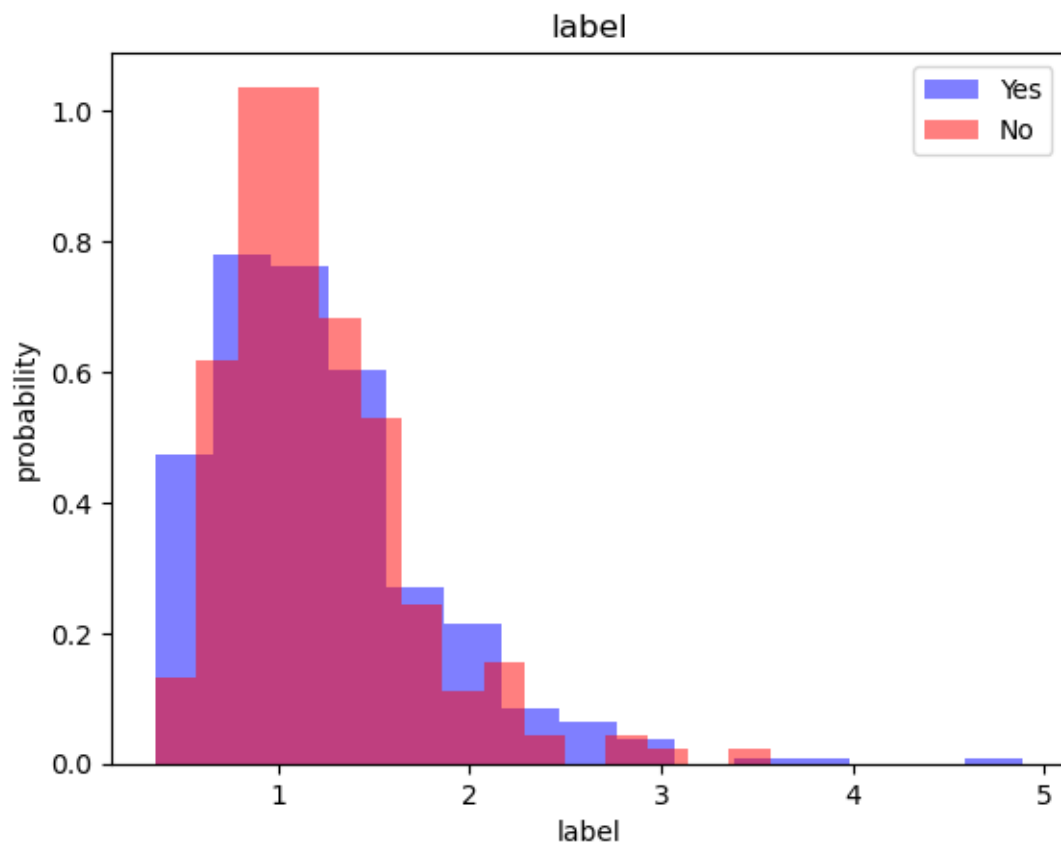


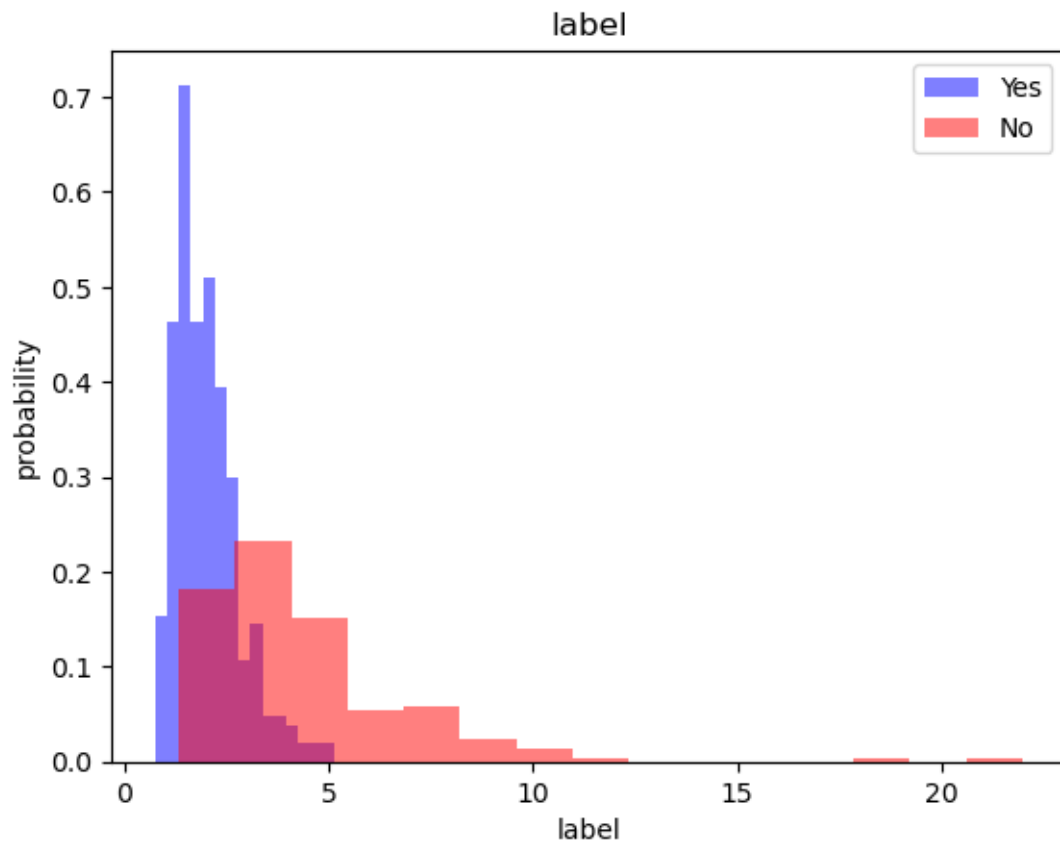


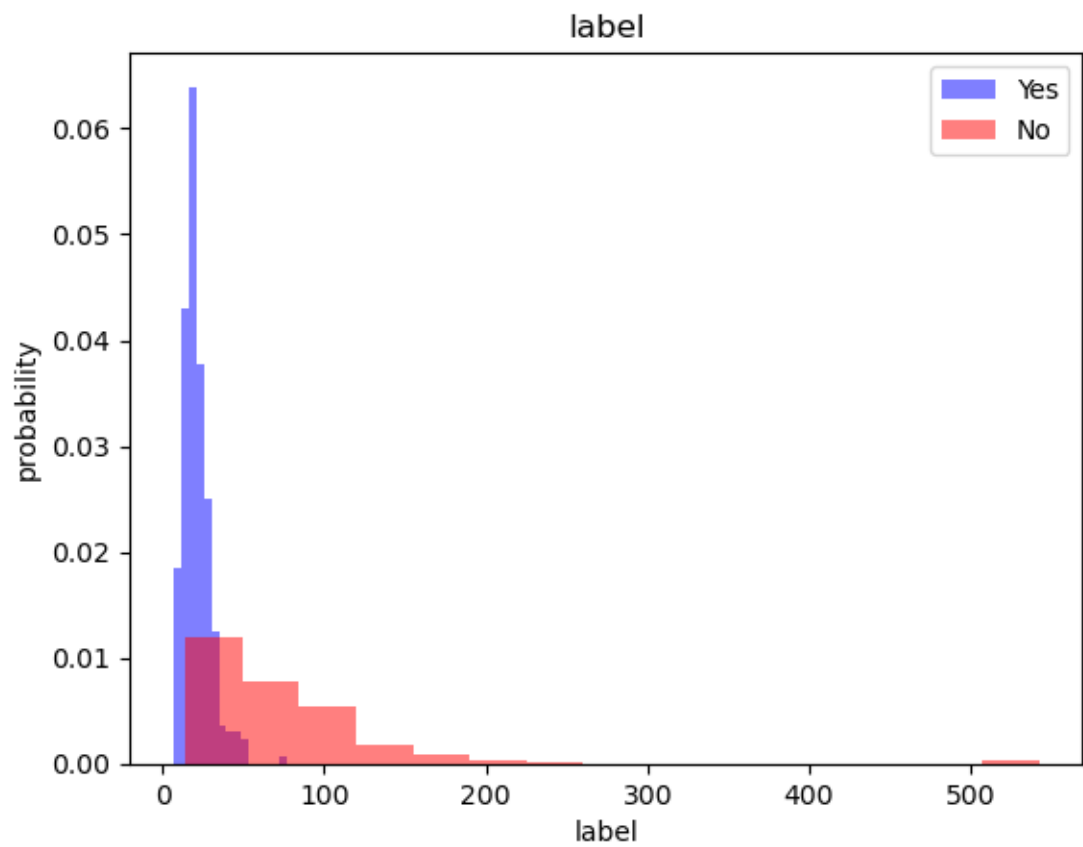


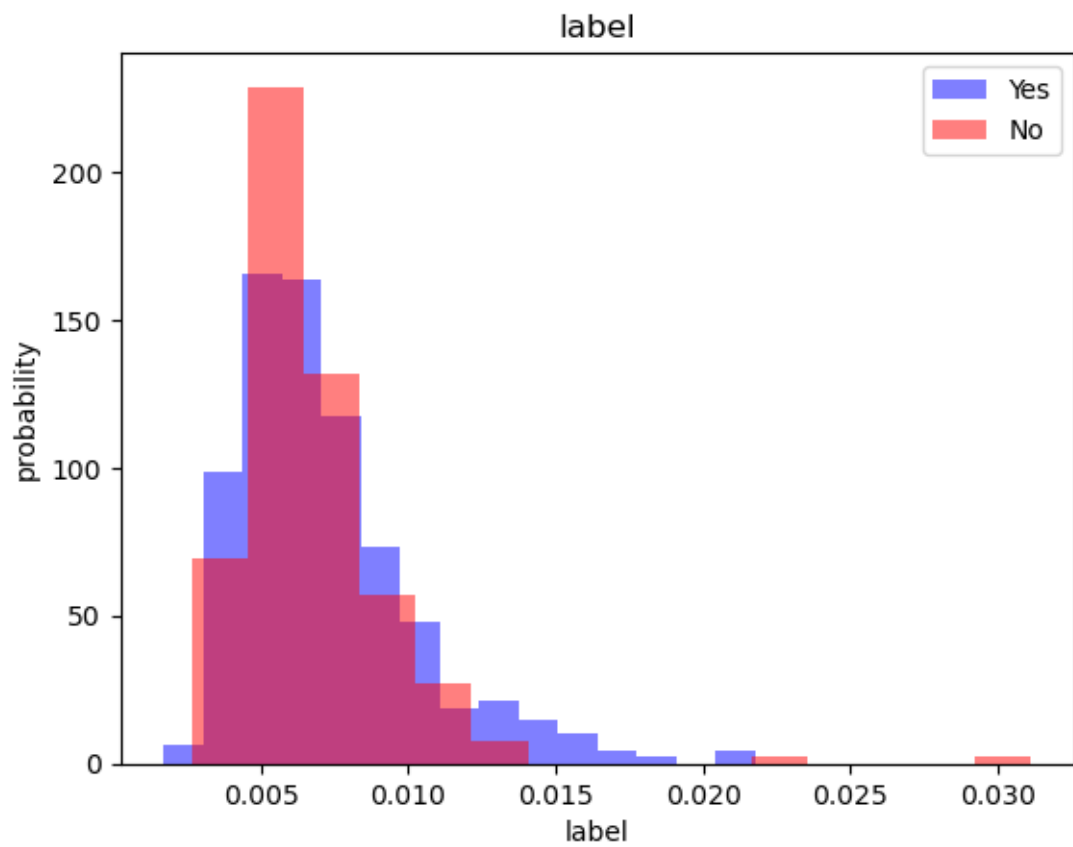


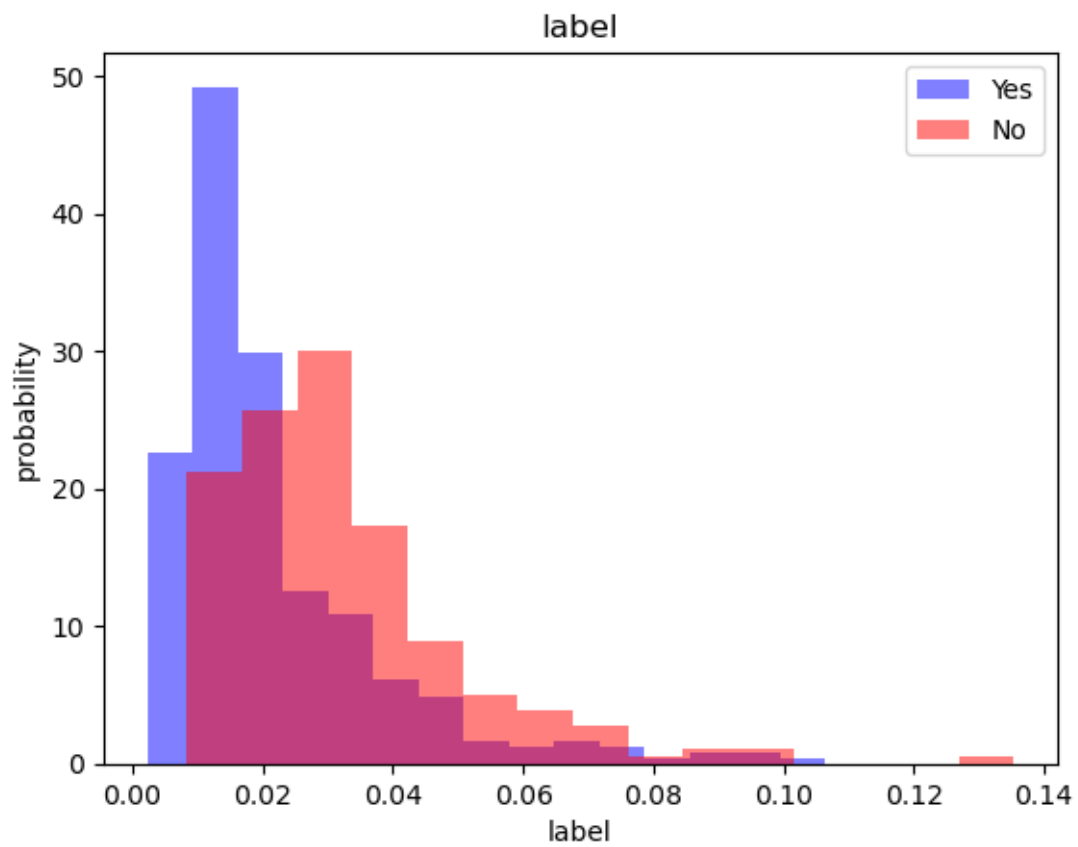


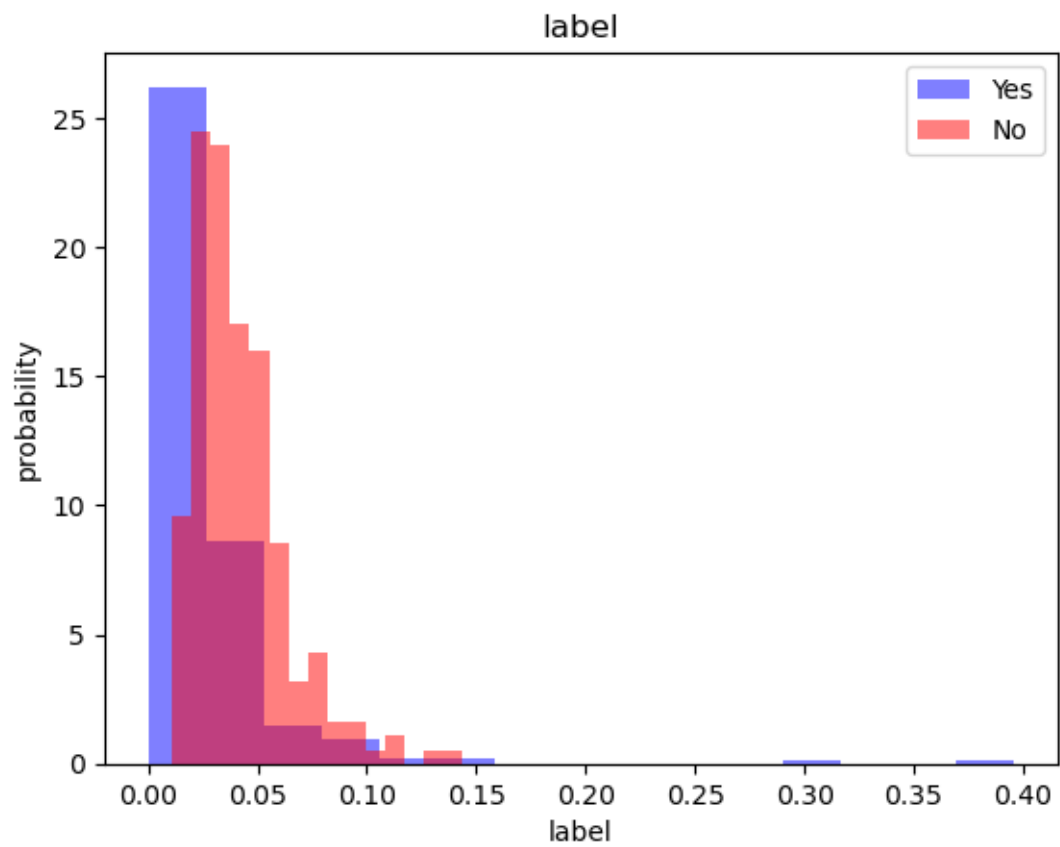


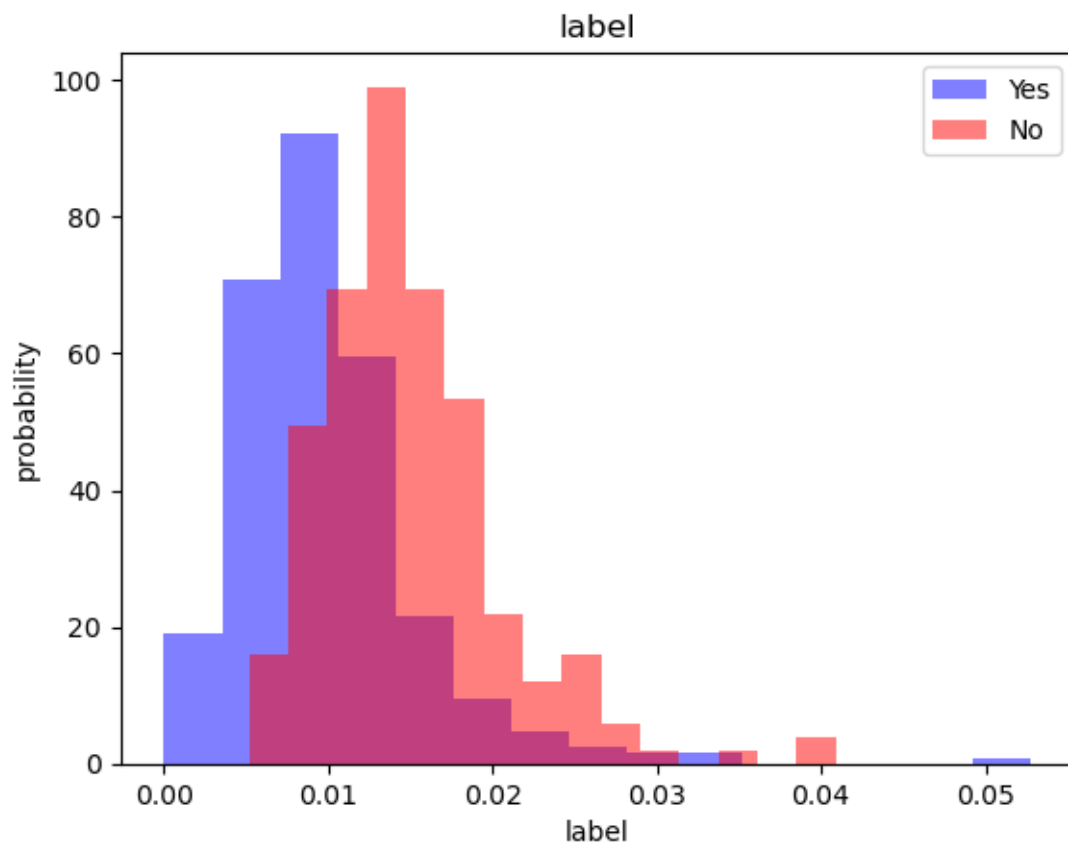


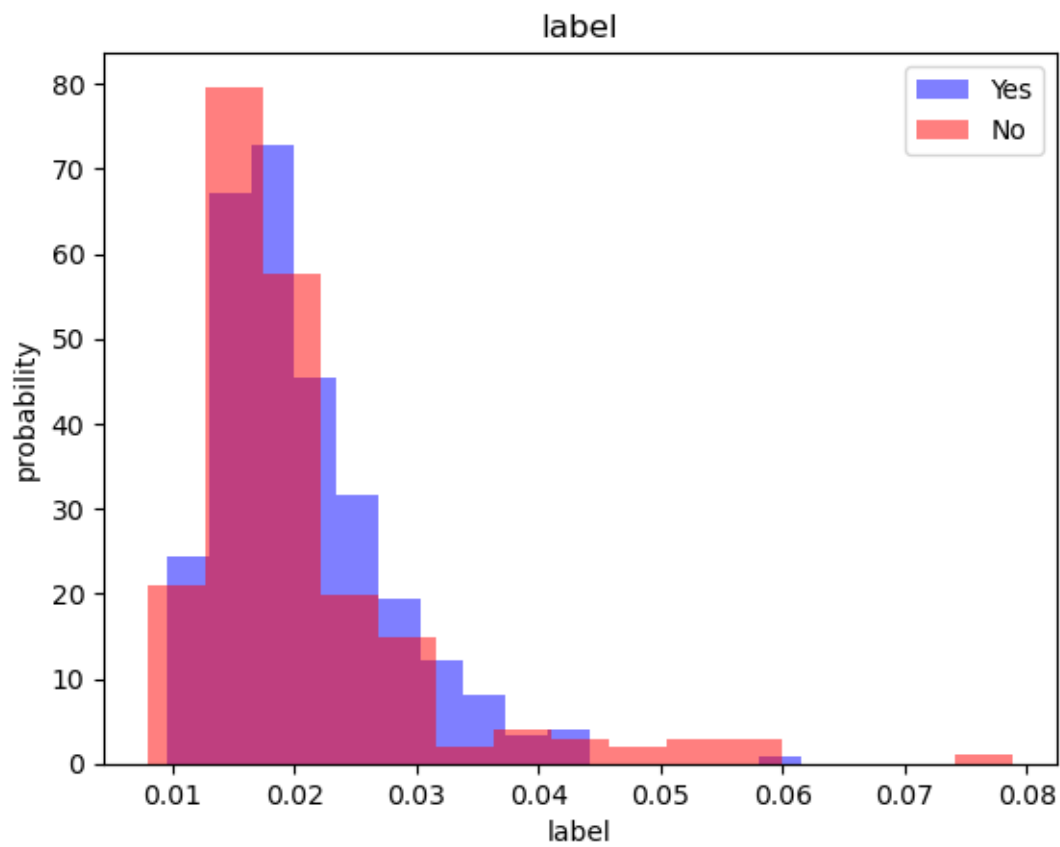


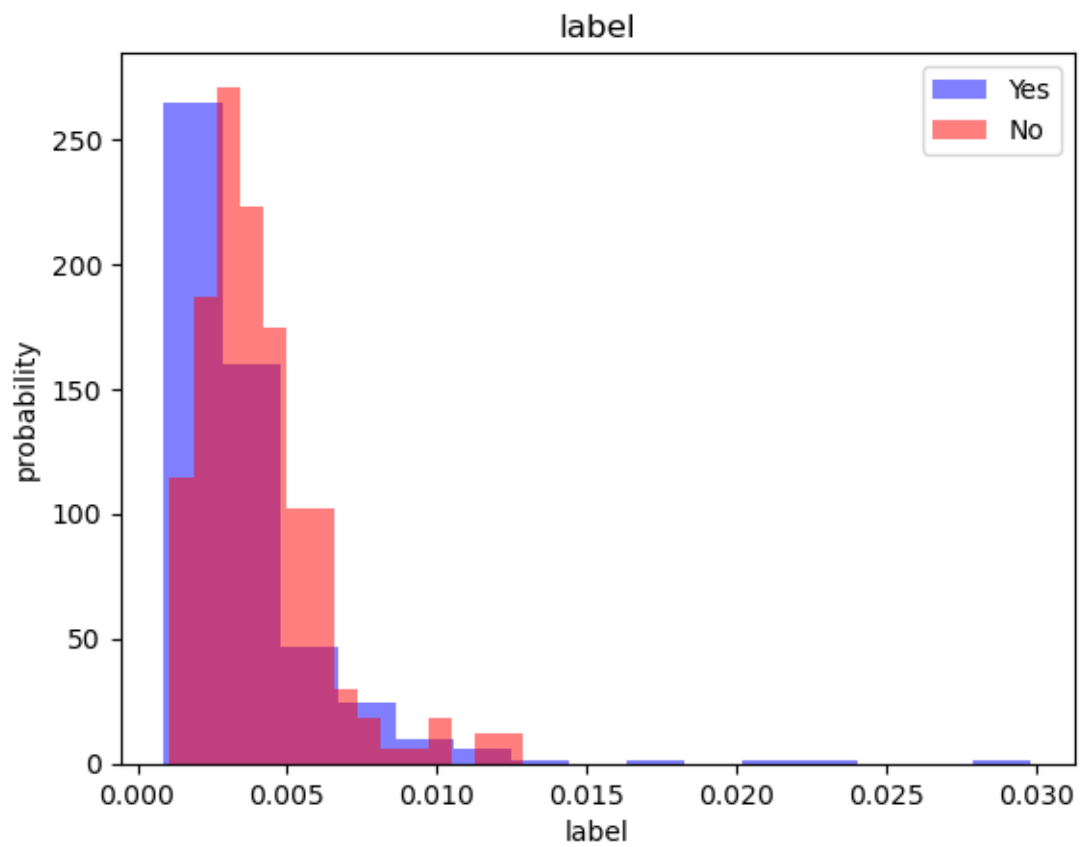


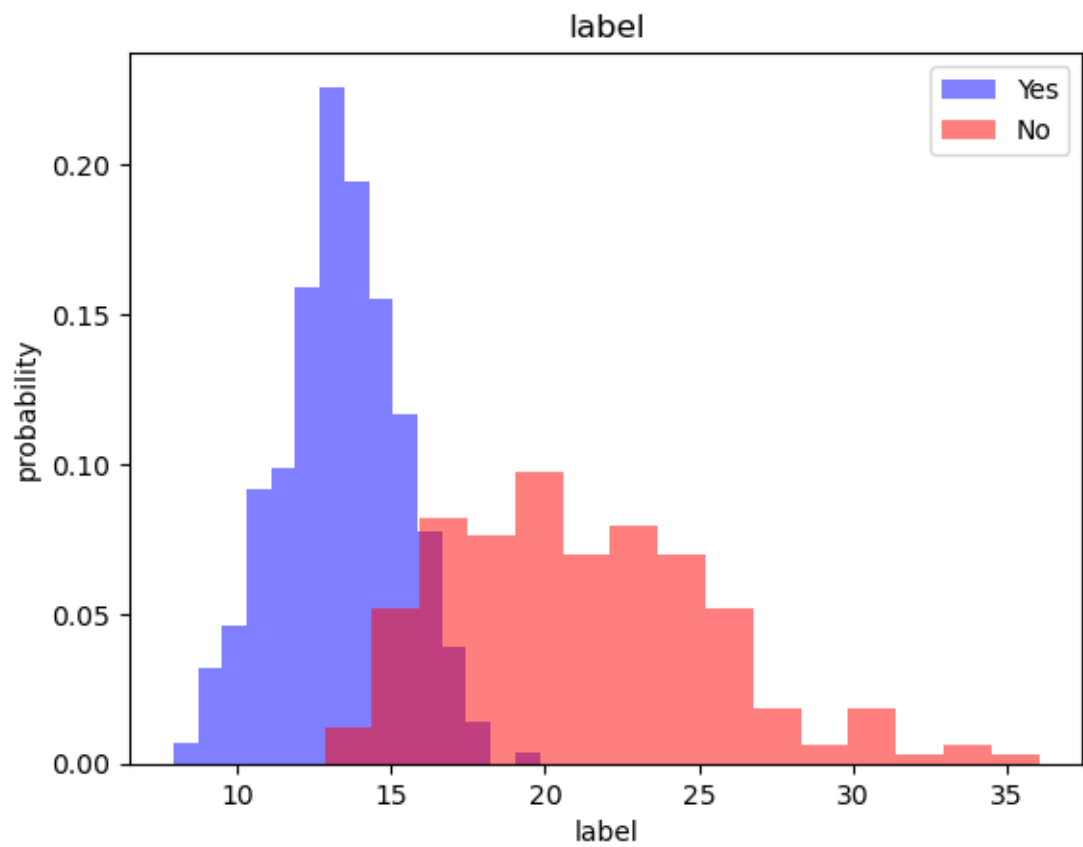


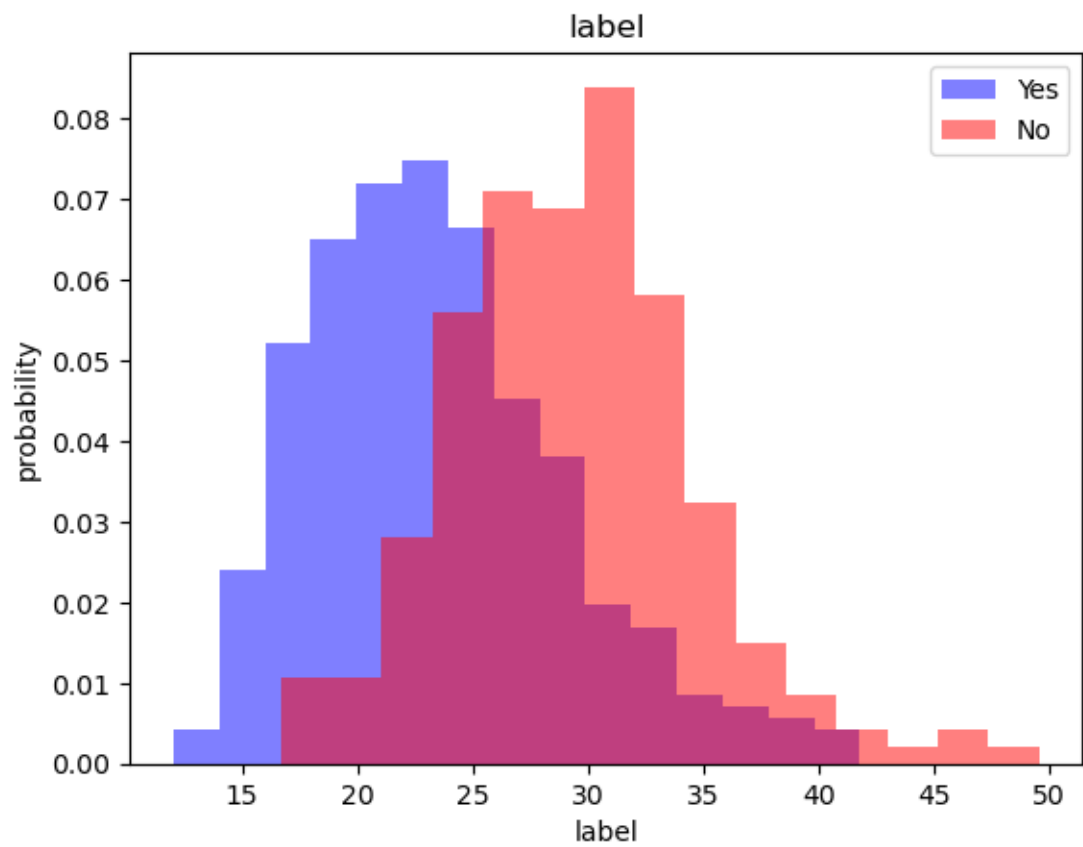


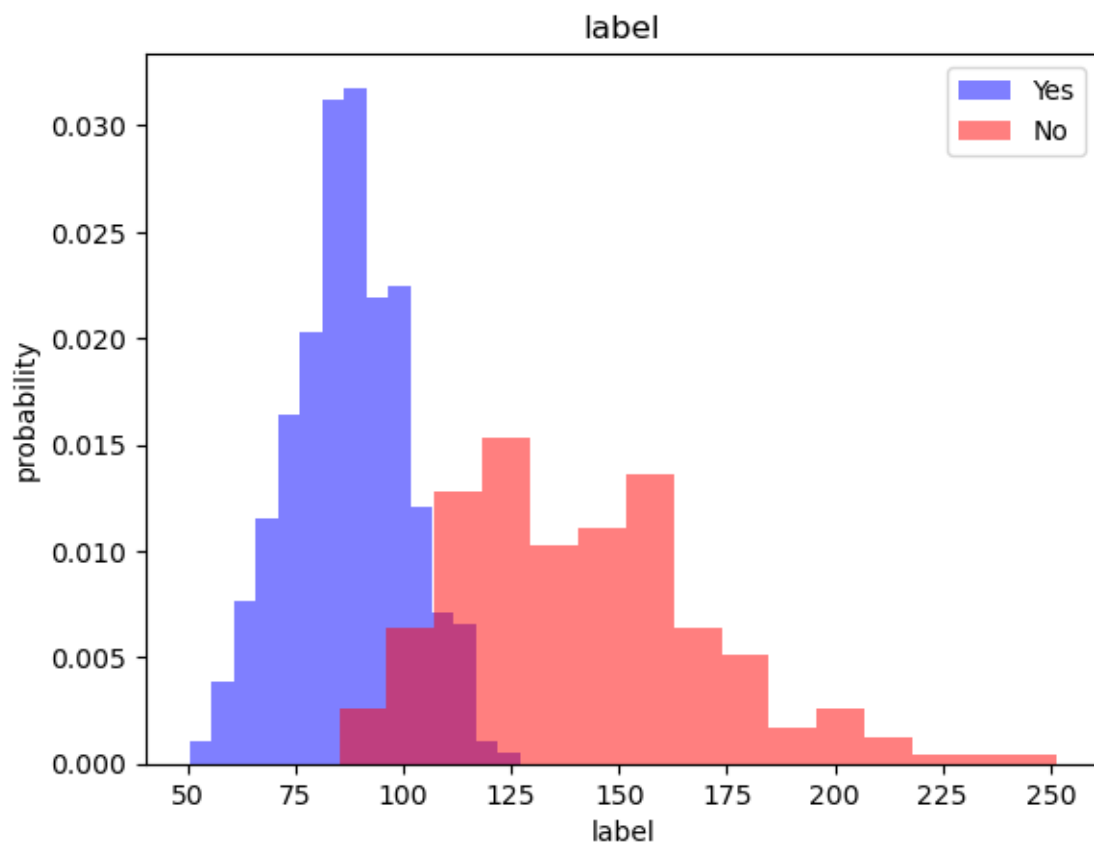


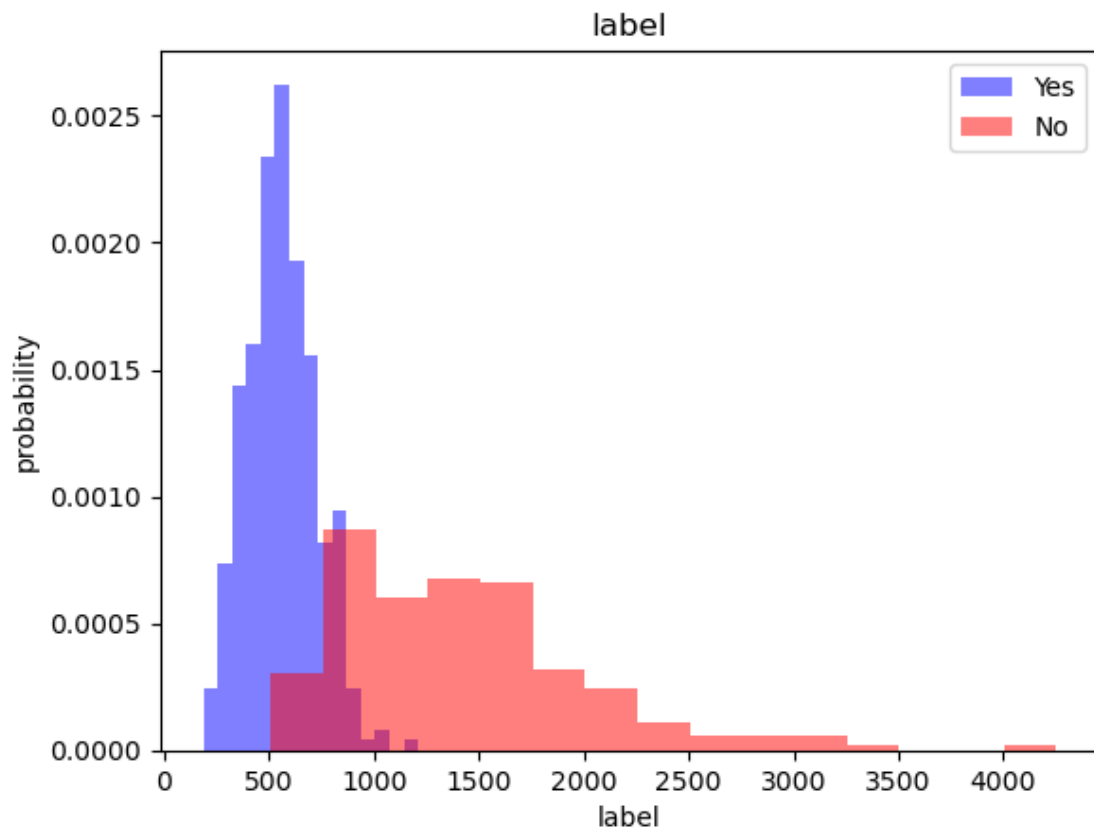


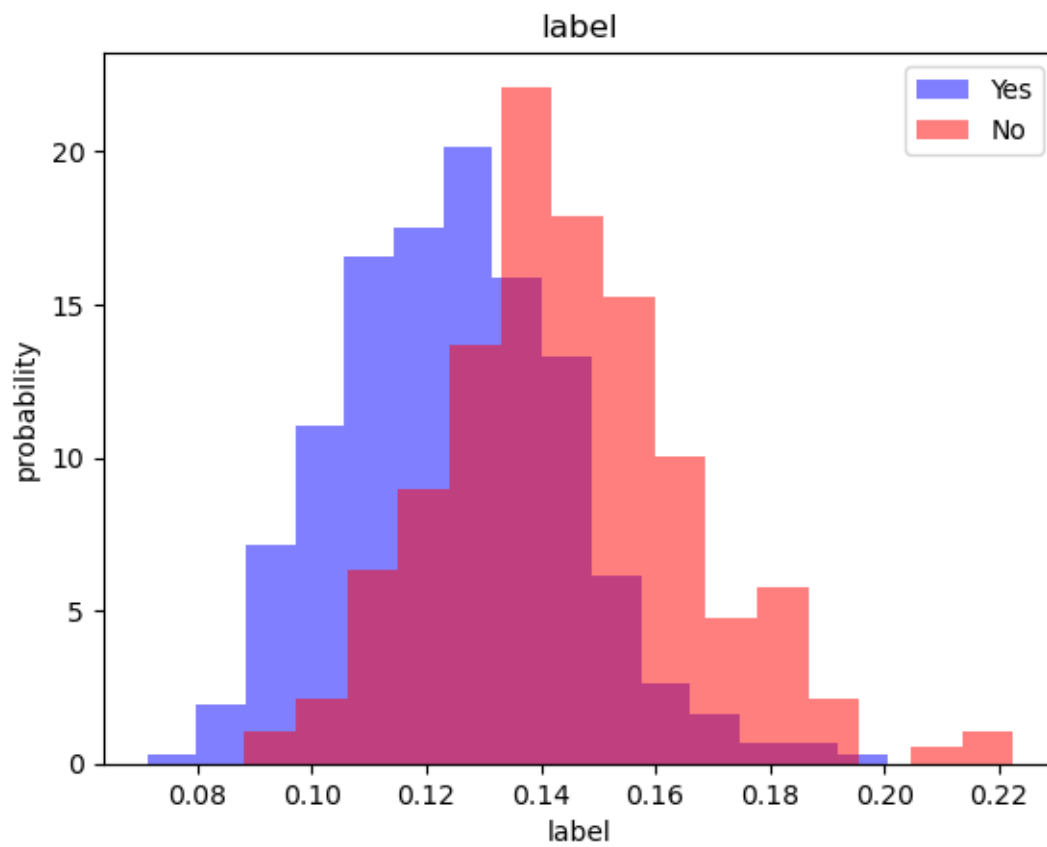


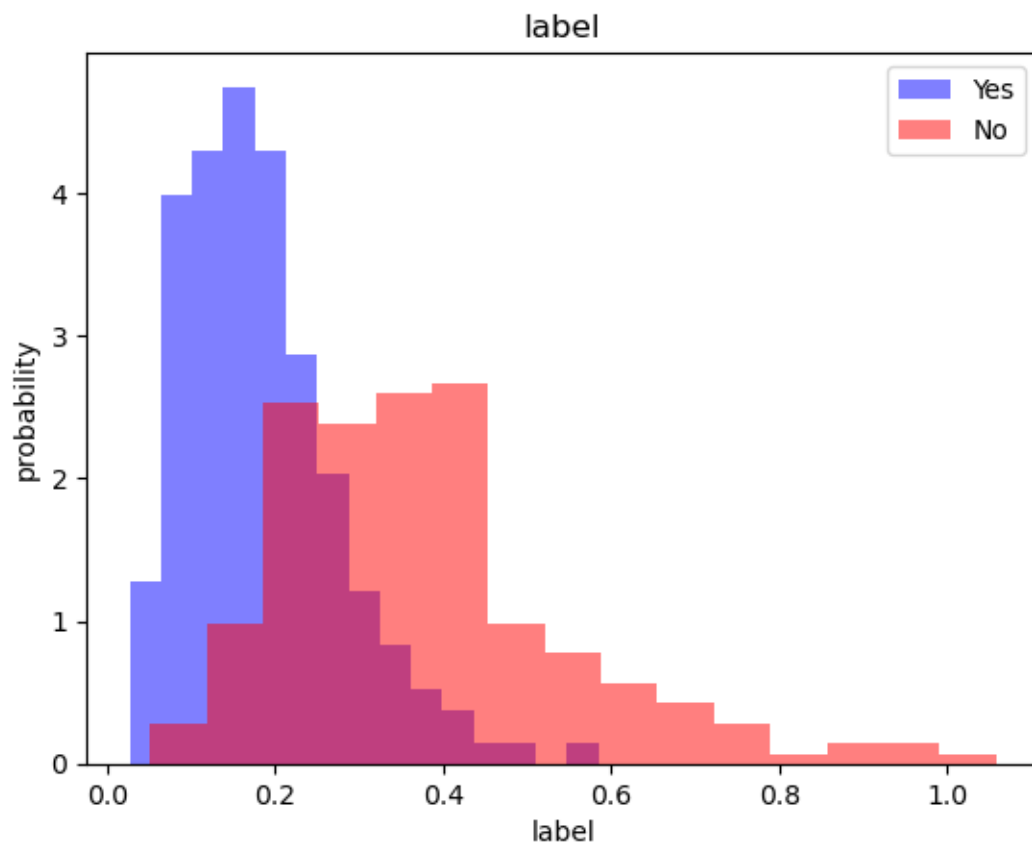


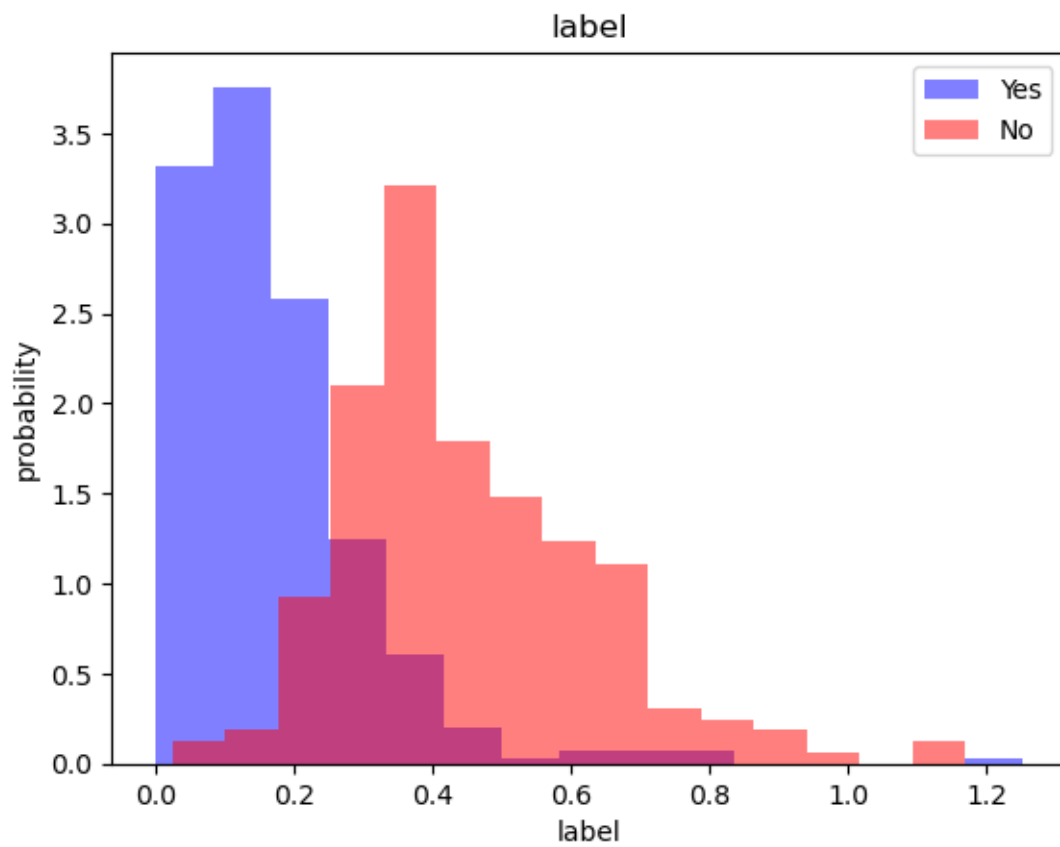


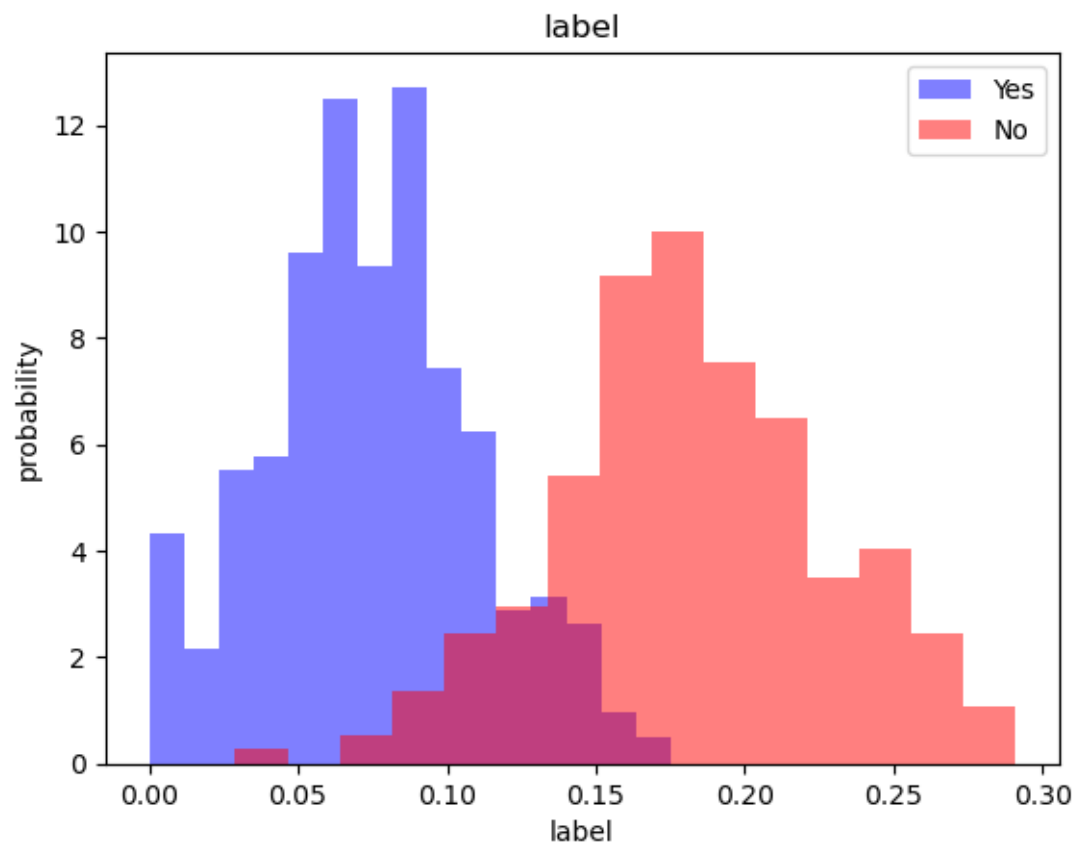


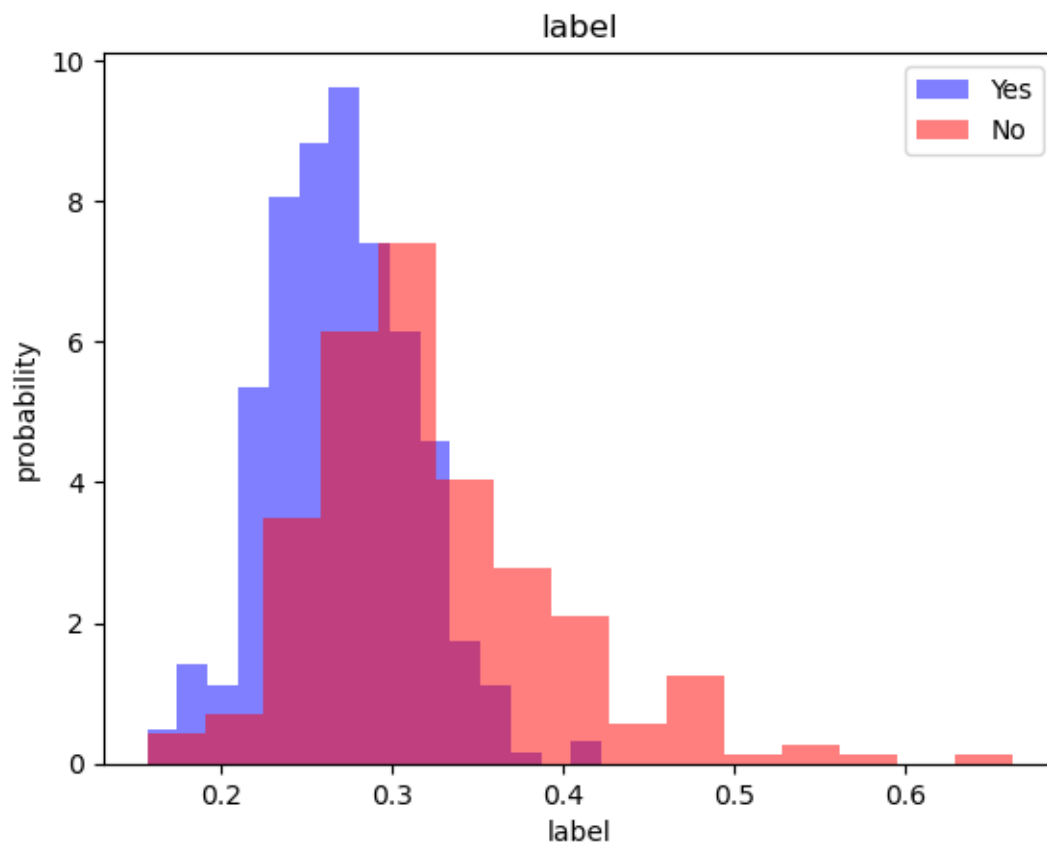


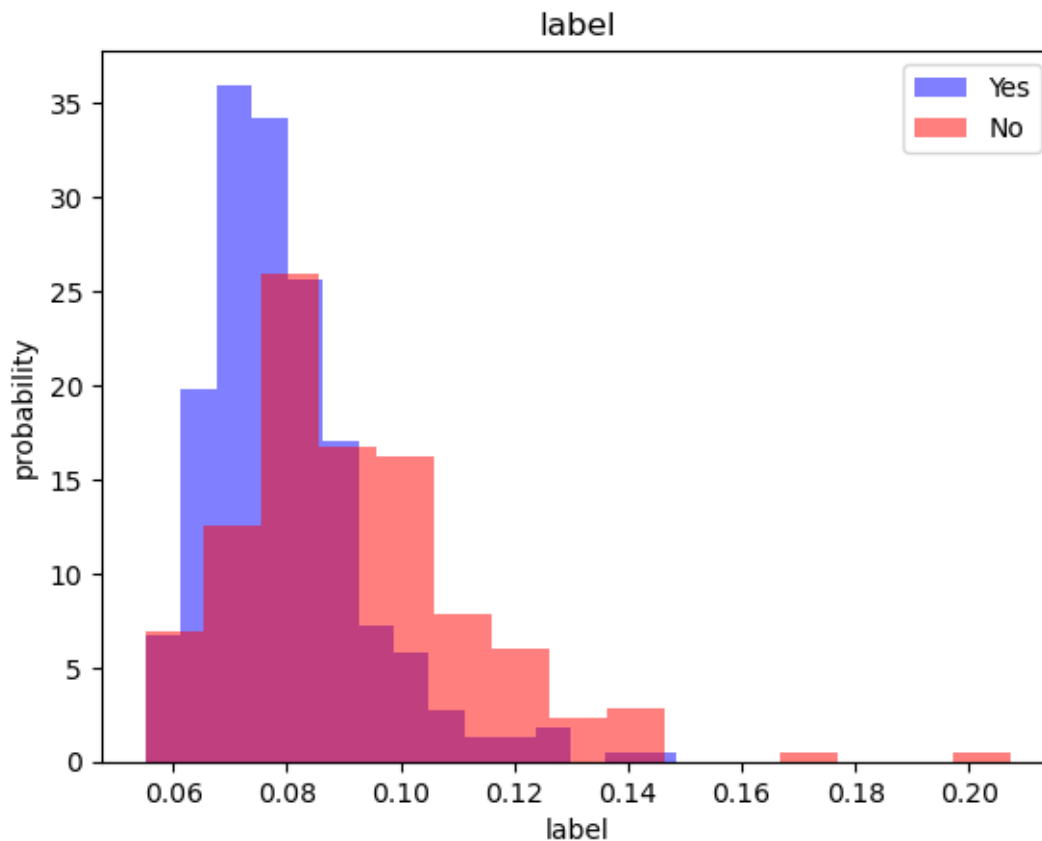












```
[13]: from sklearn.model_selection import train_test_split
      from sklearn.preprocessing import StandardScaler
```

```
[14]: X = df.drop( "target", axis = 1) # the Feature Variable
      y = df["target"] # Target variable contains "Yes" == 1, "No" == 0
```

```
[15]: X.shape
```

```
[15]: (569, 30)
```

```
[16]: y.shape
```

```
[16]: (569,)
```

```
[17]: X.shape
```

```
[17]: (569, 30)
```

```
[18]: X_train,X_test,y_train, y_test = train_test_split(X,
                                                         y,
```

```
test_size = 0.2,
random_state =42
)
```

```
[19]: X_train
```

```
[19]:      mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
68          9.029         17.33         58.79         250.5         0.10660
181         21.090         26.57        142.70        1311.0         0.11410
63          9.173         13.86         59.20         260.9         0.07721
248         10.650         25.22         68.01         347.0         0.09657
60         10.170         14.88         64.55         311.9         0.11340
..          ...          ...          ...          ...          ...
71          8.888         14.64         58.79         244.0         0.09783
106         11.640         18.33         75.17         412.5         0.11420
270         14.290         16.82         90.30         632.6         0.06429
435         13.980         19.62         91.12         599.5         0.10600
102         12.180         20.52         77.22         458.7         0.08013

      mean compactness  mean concavity  mean concave points  mean symmetry  \
68          0.14130         0.31300         0.04375         0.2111
181         0.28320         0.24870         0.14960         0.2395
63          0.08751         0.05988         0.02180         0.2341
248         0.07234         0.02379         0.01615         0.1897
60          0.08061         0.01084         0.01290         0.2743
..          ...          ...          ...          ...
71          0.15310         0.08606         0.02872         0.1902
106         0.10170         0.07070         0.03485         0.1801
270         0.02675         0.00725         0.00625         0.1508
435         0.11330         0.11260         0.06463         0.1669
102         0.04038         0.02383         0.01770         0.1739

      mean fractal dimension  ...  worst radius  worst texture  \
68          0.08046  ...         10.310         22.65
181         0.07398  ...         26.680         33.48
63          0.06963  ...         10.010         19.23
248         0.06329  ...         12.250         35.19
60          0.06960  ...         11.020         17.45
..          ...  ...          ...          ...
71          0.08980  ...          9.733         15.67
106         0.06520  ...         13.140         29.26
270         0.05376  ...         14.910         20.65
435         0.06544  ...         17.040         30.80
102         0.05677  ...         13.340         32.84

      worst perimeter  worst area  worst smoothness  worst compactness  \
68          65.50         324.7         0.14820         0.43650
```

181	176.50	2089.0	0.14910	0.75840
63	65.59	310.1	0.09836	0.16780
248	77.98	455.7	0.14990	0.13980
60	69.86	368.6	0.12750	0.09866
..	...	...	...	...
71	62.56	284.4	0.12070	0.24360
106	85.51	521.7	0.16880	0.26600
270	94.44	684.6	0.08567	0.05036
435	113.90	869.3	0.16130	0.35680
102	84.58	547.8	0.11230	0.08862

	worst concavity	worst concave points	worst symmetry \
68	1.25200	0.17500	0.4228
181	0.67800	0.29030	0.4098
63	0.13970	0.05087	0.3282
248	0.11250	0.06136	0.3409
60	0.02168	0.02579	0.3557
..	...	...	...
71	0.14340	0.04786	0.2254
106	0.28730	0.12180	0.2806
270	0.03866	0.03333	0.2458
435	0.40690	0.18270	0.3179
102	0.11450	0.07431	0.2694

	worst fractal dimension
68	0.11750
181	0.12840
63	0.08490
248	0.08147
60	0.08020
..	...
71	0.10840
106	0.09097
270	0.06120
435	0.10550
102	0.06878

[455 rows x 30 columns]

[20]: X_test

	mean radius	mean texture	mean perimeter	mean area	mean smoothness \
204	12.47	18.60	81.09	481.9	0.09965
70	18.94	21.31	123.60	1130.0	0.09009
131	15.46	19.48	101.70	748.9	0.10920
431	12.40	17.68	81.47	467.8	0.10540
540	11.54	14.44	74.65	402.9	0.09984

..	...	...	...	...	...
486	14.64	16.85	94.21	666.0	0.08641
75	16.07	19.65	104.10	817.7	0.09168
249	11.52	14.93	73.87	406.3	0.10130
238	14.22	27.85	92.55	623.9	0.08223
265	20.73	31.12	135.70	1419.0	0.09469

	mean compactness	mean concavity	mean concave points	mean symmetry \
204	0.10580	0.08005	0.03821	0.1925
70	0.10290	0.10800	0.07951	0.1582
131	0.12230	0.14660	0.08087	0.1931
431	0.13160	0.07741	0.02799	0.1811
540	0.11200	0.06737	0.02594	0.1818
..	...	...	...	...
486	0.06698	0.05192	0.02791	0.1409
75	0.08424	0.09769	0.06638	0.1798
249	0.07808	0.04328	0.02929	0.1883
238	0.10390	0.11030	0.04408	0.1342
265	0.11430	0.13670	0.08646	0.1769

	mean fractal dimension	...	worst radius	worst texture \
204	0.06373	...	14.97	24.64
70	0.05461	...	24.86	26.58
131	0.05796	...	19.26	26.00
431	0.07102	...	12.88	22.91
540	0.06782	...	12.26	19.68
..	...	...	...	...
486	0.05355	...	16.46	25.44
75	0.05391	...	19.77	24.56
249	0.06168	...	12.65	21.19
238	0.06129	...	15.75	40.54
265	0.05674	...	32.49	47.16

	worst perimeter	worst area	worst smoothness	worst compactness \
204	96.05	677.9	0.1426	0.2378
70	165.90	1866.0	0.1193	0.2336
131	124.90	1156.0	0.1546	0.2394
431	89.61	515.8	0.1450	0.2629
540	78.78	457.8	0.1345	0.2118
..	...	...	...	...
486	106.00	831.0	0.1142	0.2070
75	128.80	1223.0	0.1500	0.2045
249	80.88	491.8	0.1389	0.1582
238	102.50	764.0	0.1081	0.2426
265	214.00	3432.0	0.1401	0.2644

	worst concavity	worst concave points	worst symmetry \
--	-----------------	----------------------	------------------

204	0.2671	0.10150	0.3014
70	0.2687	0.17890	0.2551
131	0.3791	0.15140	0.2837
431	0.2403	0.07370	0.2556
540	0.1797	0.06918	0.2329
..	...	...	...
486	0.2437	0.07828	0.2455
75	0.2829	0.15200	0.2650
249	0.1804	0.09608	0.2664
238	0.3064	0.08219	0.1890
265	0.3442	0.16590	0.2868

worst fractal dimension	
204	0.08750
70	0.06589
131	0.08019
431	0.09359
540	0.08134
..	...
486	0.06596
75	0.06387
249	0.07809
238	0.07796
265	0.08218

[114 rows x 30 columns]

```
[21]: y_train,y_test
```

```
[21]: (68      1
      181     0
      63      1
      248     1
      60      1
      ..
      71      1
      106     1
      270     1
      435     0
      102     1
      Name: target, Length: 455, dtype: int64,
      204     1
      70      0
      131     0
      431     1
      540     1
      ..)
```

```

486    1
75     0
249    1
238    1
265    0
Name: target, Length: 114, dtype: int64)

```

```

[22]: scaler = StandardScaler()
X_trained_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.fit_transform(X_test)

```

```

[23]: from sklearn.linear_model import LogisticRegression
lr = LogisticRegression()
lr.fit(X_trained_scaled,y_train)
y_pred = lr.predict(X_test_scaled)

```

```

[24]: y_pred

```

```

[24]: array([1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1,
          0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1,
          1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1,
          0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0,
          1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1,
          0, 1, 1, 0])

```

```

[25]: print(y.value_counts())

```

```

target
1    357
0    212
Name: count, dtype: int64

```

```

[26]: model = LinearRegression()
model.fit(X_trained_scaled,y_train)

predictions = model.predict(X_test_scaled)
predictions

```

```

[26]: array([ 0.63968721,  0.22740226,  0.32361512,  1.200943   ,  1.19426351,
          -0.45850344,  0.03507188,  0.51100044,  0.34253912,  0.96568587,
           0.8023944 ,  0.37382778,  0.89913764,  0.5195883 ,  0.91616539,
           0.25508887,  0.9082683 ,  1.14003361,  1.33145512,  0.14643982,
           0.66789428,  0.86898586, -0.24725593,  1.03923885,  0.95661002,
           0.82858602,  1.03646754,  0.92965204,  0.93311782, -0.03928239,
           1.06383079,  1.0202386 ,  0.73468352,  0.89497087,  1.02367197,
           0.80975679,  0.49603914,  0.57409927,  0.29801476,  0.76982672,
           1.1605838 ,  0.11940268,  0.97874026,  1.01047958,  0.50803949,
           0.66681999,  0.85480572,  0.96623662,  0.81058351,  1.17373593,

```

```

0.24346832, 0.05803547, 0.58223574, 0.63661059, 1.0391142 ,
0.79723021, 0.9755658 , -0.17773385, 0.60053297, 1.02399552,
0.95398461, -0.10452604, -0.22631651, 0.83895359, 0.87293584,
0.63006452, -0.03840662, 0.06758752, 0.96062681, 0.89850115,
0.14113892, 0.15386398, 0.84481598, 0.37501308, 0.92137147,
0.82740334, 0.78402196, 0.61992903, 1.18592802, 0.74912319,
0.44056318, 1.01108601, 0.61502411, 0.11565107, 0.35401415,
0.22506261, -0.17239285, 0.20463834, 1.05970704, 0.9494094 ,
0.82553472, 0.78431158, 0.82323408, 1.07999724, 0.92211558,
0.94742323, 0.0980465 , -0.16009551, 0.99463806, 0.24787306,
0.28363121, 1.21732617, 0.31330849, 0.20591948, 0.73814434,
0.72881162, 0.83685141, -0.01237638, 0.74713578, 0.77607129,
0.20715039, 0.96434081, 0.65350327, 0.21373627])

```

```

[27]: residual = y_test - predictions
      residual

```

```

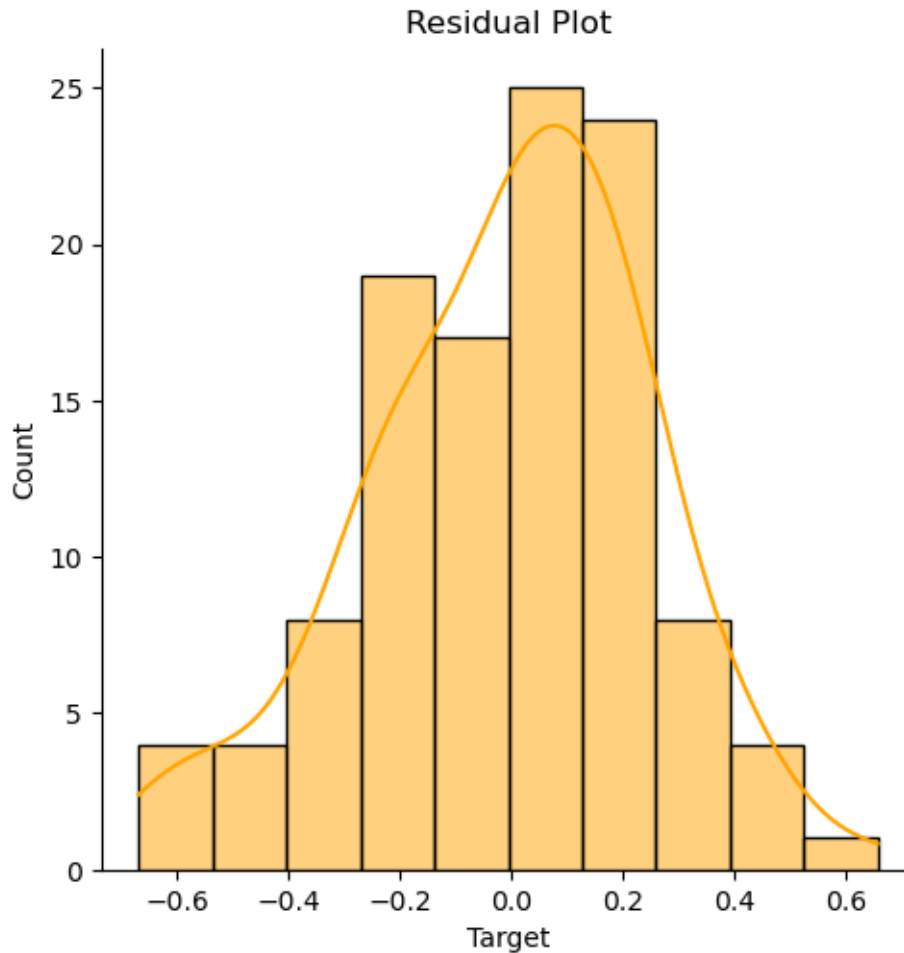
[27]: 204    0.360313
      70    -0.227402
      131   -0.323615
      431   -0.200943
      540   -0.194264
      ...
      486    0.223929
      75    -0.207150
      249    0.035659
      238    0.346497
      265   -0.213736
      Name: target, Length: 114, dtype: float64

```

```

[28]: sns.displot(residual, kde =True, bins =10, color = "orange")
      plt.title("Residual Plot")
      plt.xlabel("Target")
      plt.ylabel("Count")
      plt.show()

```



```
[29]: from sklearn.metrics import mean_squared_error, mean_absolute_error
import math
```

```
[30]: print (f"Mean Squared Errorr: {mean_squared_error(y_test,y_pred)}")
print (f"Mean Absolute Error : {mean_absolute_error(y_test,y_pred)}")
print (f"Mean Sqrt Absolute Errorr: {math.
↪sqrt(mean_absolute_error(y_test,y_pred))}")
```

Mean Squared Errorr: 0.017543859649122806

Mean Absolute Error : 0.017543859649122806

Mean Sqrt Absolute Errorr: 0.13245323570650439

```
[31]: !pip install tensorflow
```

Requirement already satisfied: tensorflow in c:\users\user\anaconda3\lib\site-packages (2.21.0)

Requirement already satisfied: absl-py>=1.0.0 in

c:\users\user\anaconda3\lib\site-packages (from tensorflow) (2.5.0)

Requirement already satisfied: astunparse>=1.6.0 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (1.6.3)

Requirement already satisfied: flatbuffers>=25.9.23 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (25.12.19)

Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (0.7.0)

Requirement already satisfied: google_pasta>=0.1.1 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (0.2.0)

Requirement already satisfied: libclang>=13.0.0 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (18.1.1)

Requirement already satisfied: opt_einsum>=2.3.2 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (3.4.0)

Requirement already satisfied: packaging in c:\users\user\anaconda3\lib\site-packages (from tensorflow) (25.0)

Requirement already satisfied: protobuf<8.0.0,>=6.31.1 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (7.35.1)

Requirement already satisfied: requests<3,>=2.21.0 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (2.34.2)

Requirement already satisfied: setuptools in c:\users\user\anaconda3\lib\site-packages (from tensorflow) (80.9.0)

Requirement already satisfied: six>=1.12.0 in c:\users\user\anaconda3\lib\site-packages (from tensorflow) (1.17.0)

Requirement already satisfied: termcolor>=1.1.0 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (3.3.0)

Requirement already satisfied: typing_extensions>=3.6.6 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (4.15.0)

Requirement already satisfied: wrapt>=1.11.0 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (1.17.0)

Requirement already satisfied: grpcio<2.0,>=1.24.3 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (1.84.0)

Requirement already satisfied: keras>=3.12.0 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (3.15.1)

Requirement already satisfied: numpy>=1.26.0 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (2.3.5)

Requirement already satisfied: h5py<3.15.0,>=3.11.0 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (3.14.0)

Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in
c:\users\user\anaconda3\lib\site-packages (from tensorflow) (0.6.0)

Requirement already satisfied: charset_normalizer<4,>=2 in
c:\users\user\anaconda3\lib\site-packages (from requests<3,>=2.21.0->tensorflow) (3.4.4)

Requirement already satisfied: idna<4,>=2.5 in c:\users\user\anaconda3\lib\site-packages (from requests<3,>=2.21.0->tensorflow) (3.11)

Requirement already satisfied: urllib3<3,>=1.26 in
c:\users\user\anaconda3\lib\site-packages (from requests<3,>=2.21.0->tensorflow) (2.5.0)

Requirement already satisfied: certifi>=2023.5.7 in
c:\users\user\anaconda3\lib\site-packages (from requests<3,>=2.21.0->tensorflow)

(2026.5.20)

Requirement already satisfied: wheel<1.0,>=0.23.0 in
c:\users\user\anaconda3\lib\site-packages (from astunparse>=1.6.0->tensorflow)
(0.45.1)
Requirement already satisfied: rich in c:\users\user\anaconda3\lib\site-packages
(from keras>=3.12.0->tensorflow) (14.2.0)
Requirement already satisfied: namex in c:\users\user\anaconda3\lib\site-
packages (from keras>=3.12.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in c:\users\user\anaconda3\lib\site-
packages (from keras>=3.12.0->tensorflow) (0.20.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in
c:\users\user\anaconda3\lib\site-packages (from rich->keras>=3.12.0->tensorflow)
(2.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
c:\users\user\anaconda3\lib\site-packages (from rich->keras>=3.12.0->tensorflow)
(2.19.2)
Requirement already satisfied: mdurl~=0.1 in c:\users\user\anaconda3\lib\site-
packages (from markdown-it-py>=2.2.0->rich->keras>=3.12.0->tensorflow) (0.1.2)

```
[32]: import tensorflow as tf
      from tensorflow import keras
```

```
[33]: models = keras.Sequential([
      keras.layers.Dense(80,activation= "relu",input_shape=(30,)),
      keras.layers.Dense(40,activation= "relu"),
      keras.layers.Dense(1, activation= "sigmoid")
      ])
```

C:\Users\User\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:107:
UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When
using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
[34]: # compile the model
      models.compile(optimizer = "adam",
      loss = "binary_crossentropy",
      metrics =["accuracy"])
      #lets fit into the model
      history= models.fit(X_trained_scaled,
      y_train,
      validation_split = 0.2,
      verbose = 1,
      epochs = 100,
      batch_size = 2)
```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used.

Please use WSL2 or the TensorFlow-DirectML plugin.

Epoch 1/100

182/182 4s 7ms/step -

accuracy: 0.9313 - loss: 0.1813 - val_accuracy: 0.9560 - val_loss: 0.1096

Epoch 2/100

182/182 1s 5ms/step -

accuracy: 0.9863 - loss: 0.0663 - val_accuracy: 0.9560 - val_loss: 0.0986

Epoch 3/100

182/182 1s 5ms/step -

accuracy: 0.9835 - loss: 0.0467 - val_accuracy: 0.9670 - val_loss: 0.0829

Epoch 4/100

182/182 1s 4ms/step -

accuracy: 0.9890 - loss: 0.0353 - val_accuracy: 0.9670 - val_loss: 0.0765

Epoch 5/100

182/182 1s 4ms/step -

accuracy: 0.9945 - loss: 0.0280 - val_accuracy: 0.9560 - val_loss: 0.0806

Epoch 6/100

182/182 1s 4ms/step -

accuracy: 0.9945 - loss: 0.0201 - val_accuracy: 0.9560 - val_loss: 0.0892

Epoch 7/100

182/182 1s 4ms/step -

accuracy: 0.9945 - loss: 0.0168 - val_accuracy: 0.9560 - val_loss: 0.0999

Epoch 8/100

182/182 1s 4ms/step -

accuracy: 0.9945 - loss: 0.0155 - val_accuracy: 0.9670 - val_loss: 0.0950

Epoch 9/100

182/182 1s 4ms/step -

accuracy: 0.9945 - loss: 0.0100 - val_accuracy: 0.9451 - val_loss: 0.1006

Epoch 10/100

182/182 1s 4ms/step -

accuracy: 0.9973 - loss: 0.0076 - val_accuracy: 0.9560 - val_loss: 0.1267

Epoch 11/100

182/182 1s 4ms/step -

accuracy: 1.0000 - loss: 0.0058 - val_accuracy: 0.9670 - val_loss: 0.1142

Epoch 12/100

182/182 1s 5ms/step -

accuracy: 1.0000 - loss: 0.0054 - val_accuracy: 0.9670 - val_loss: 0.0979

Epoch 13/100

182/182 1s 4ms/step -

accuracy: 1.0000 - loss: 0.0028 - val_accuracy: 0.9451 - val_loss: 0.1090

Epoch 14/100

182/182 1s 4ms/step -

accuracy: 1.0000 - loss: 0.0029 - val_accuracy: 0.9560 - val_loss: 0.1124

Epoch 15/100

182/182 1s 4ms/step -

accuracy: 1.0000 - loss: 0.0017 - val_accuracy: 0.9670 - val_loss: 0.1204

Epoch 16/100

182/182 1s 4ms/step -

accuracy: 1.0000 - loss: 0.0015 - val_accuracy: 0.9451 - val_loss: 0.0982
 Epoch 17/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 0.0013 - val_accuracy: 0.9560 - val_loss: 0.1167
 Epoch 18/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 0.0012 - val_accuracy: 0.9560 - val_loss: 0.1389
 Epoch 19/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 7.3149e-04 - val_accuracy: 0.9560 - val_loss: 0.1285
 Epoch 20/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 5.8246e-04 - val_accuracy: 0.9560 - val_loss: 0.1301
 Epoch 21/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 4.8977e-04 - val_accuracy: 0.9560 - val_loss: 0.1266
 Epoch 22/100
 182/182 1s 5ms/step -
 accuracy: 1.0000 - loss: 4.0119e-04 - val_accuracy: 0.9560 - val_loss: 0.1356
 Epoch 23/100
 182/182 1s 5ms/step -
 accuracy: 1.0000 - loss: 3.7480e-04 - val_accuracy: 0.9560 - val_loss: 0.1292
 Epoch 24/100
 182/182 1s 5ms/step -
 accuracy: 1.0000 - loss: 2.9387e-04 - val_accuracy: 0.9560 - val_loss: 0.1309
 Epoch 25/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 2.6522e-04 - val_accuracy: 0.9560 - val_loss: 0.1317
 Epoch 26/100
 182/182 1s 5ms/step -
 accuracy: 1.0000 - loss: 2.1500e-04 - val_accuracy: 0.9560 - val_loss: 0.1352
 Epoch 27/100
 182/182 1s 5ms/step -
 accuracy: 1.0000 - loss: 1.8576e-04 - val_accuracy: 0.9560 - val_loss: 0.1375
 Epoch 28/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 1.5890e-04 - val_accuracy: 0.9560 - val_loss: 0.1457
 Epoch 29/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 1.3918e-04 - val_accuracy: 0.9560 - val_loss: 0.1474
 Epoch 30/100
 182/182 1s 5ms/step -
 accuracy: 1.0000 - loss: 1.2284e-04 - val_accuracy: 0.9560 - val_loss: 0.1505
 Epoch 31/100
 182/182 1s 5ms/step -
 accuracy: 1.0000 - loss: 1.0065e-04 - val_accuracy: 0.9560 - val_loss: 0.1498
 Epoch 32/100
 182/182 1s 4ms/step -

accuracy: 1.0000 - loss: 9.5508e-05 - val_accuracy: 0.9560 - val_loss: 0.1485
 Epoch 33/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 8.2264e-05 - val_accuracy: 0.9560 - val_loss: 0.1538
 Epoch 34/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 6.6779e-05 - val_accuracy: 0.9670 - val_loss: 0.1523
 Epoch 35/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 7.4614e-05 - val_accuracy: 0.9560 - val_loss: 0.1568
 Epoch 36/100
 182/182 1s 5ms/step -
 accuracy: 1.0000 - loss: 5.2662e-05 - val_accuracy: 0.9560 - val_loss: 0.1604
 Epoch 37/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 4.9429e-05 - val_accuracy: 0.9560 - val_loss: 0.1596
 Epoch 38/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 4.1067e-05 - val_accuracy: 0.9560 - val_loss: 0.1637
 Epoch 39/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 3.9626e-05 - val_accuracy: 0.9560 - val_loss: 0.1744
 Epoch 40/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 3.5585e-05 - val_accuracy: 0.9670 - val_loss: 0.1672
 Epoch 41/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 2.9962e-05 - val_accuracy: 0.9560 - val_loss: 0.1677
 Epoch 42/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 2.6870e-05 - val_accuracy: 0.9670 - val_loss: 0.1670
 Epoch 43/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 2.2356e-05 - val_accuracy: 0.9560 - val_loss: 0.1669
 Epoch 44/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 2.1243e-05 - val_accuracy: 0.9560 - val_loss: 0.1733
 Epoch 45/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 2.0071e-05 - val_accuracy: 0.9560 - val_loss: 0.1711
 Epoch 46/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 1.6136e-05 - val_accuracy: 0.9560 - val_loss: 0.1828
 Epoch 47/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 1.5776e-05 - val_accuracy: 0.9560 - val_loss: 0.1853
 Epoch 48/100
 182/182 1s 4ms/step -

accuracy: 1.0000 - loss: 1.2921e-05 - val_accuracy: 0.9560 - val_loss: 0.1870
 Epoch 49/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 1.2320e-05 - val_accuracy: 0.9560 - val_loss: 0.1903
 Epoch 50/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 1.0250e-05 - val_accuracy: 0.9670 - val_loss: 0.1844
 Epoch 51/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 9.5948e-06 - val_accuracy: 0.9560 - val_loss: 0.1831
 Epoch 52/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 8.7496e-06 - val_accuracy: 0.9560 - val_loss: 0.1871
 Epoch 53/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 7.7343e-06 - val_accuracy: 0.9560 - val_loss: 0.1902
 Epoch 54/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 6.9431e-06 - val_accuracy: 0.9560 - val_loss: 0.1976
 Epoch 55/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 6.3729e-06 - val_accuracy: 0.9560 - val_loss: 0.1928
 Epoch 56/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 5.5010e-06 - val_accuracy: 0.9560 - val_loss: 0.2002
 Epoch 57/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 5.0782e-06 - val_accuracy: 0.9670 - val_loss: 0.2050
 Epoch 58/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 4.4484e-06 - val_accuracy: 0.9670 - val_loss: 0.2067
 Epoch 59/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 4.2587e-06 - val_accuracy: 0.9560 - val_loss: 0.2090
 Epoch 60/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 3.6133e-06 - val_accuracy: 0.9560 - val_loss: 0.2050
 Epoch 61/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 3.1358e-06 - val_accuracy: 0.9560 - val_loss: 0.2061
 Epoch 62/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 2.7744e-06 - val_accuracy: 0.9670 - val_loss: 0.2113
 Epoch 63/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 2.5594e-06 - val_accuracy: 0.9560 - val_loss: 0.2036
 Epoch 64/100
 182/182 1s 4ms/step -

accuracy: 1.0000 - loss: 2.2186e-06 - val_accuracy: 0.9670 - val_loss: 0.2115
 Epoch 65/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 2.3507e-06 - val_accuracy: 0.9670 - val_loss: 0.2114
 Epoch 66/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 2.3254e-06 - val_accuracy: 0.9560 - val_loss: 0.2220
 Epoch 67/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 1.6413e-06 - val_accuracy: 0.9670 - val_loss: 0.2198
 Epoch 68/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 1.4794e-06 - val_accuracy: 0.9670 - val_loss: 0.2205
 Epoch 69/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 1.3116e-06 - val_accuracy: 0.9560 - val_loss: 0.2254
 Epoch 70/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 1.2617e-06 - val_accuracy: 0.9560 - val_loss: 0.2215
 Epoch 71/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 1.0719e-06 - val_accuracy: 0.9670 - val_loss: 0.2300
 Epoch 72/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 9.4695e-07 - val_accuracy: 0.9670 - val_loss: 0.2301
 Epoch 73/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 8.8761e-07 - val_accuracy: 0.9670 - val_loss: 0.2262
 Epoch 74/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 7.6127e-07 - val_accuracy: 0.9670 - val_loss: 0.2309
 Epoch 75/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 7.5326e-07 - val_accuracy: 0.9560 - val_loss: 0.2399
 Epoch 76/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 6.7982e-07 - val_accuracy: 0.9670 - val_loss: 0.2413
 Epoch 77/100
 182/182 1s 3ms/step -
 accuracy: 1.0000 - loss: 5.9797e-07 - val_accuracy: 0.9560 - val_loss: 0.2323
 Epoch 78/100
 182/182 1s 3ms/step -
 accuracy: 1.0000 - loss: 5.2190e-07 - val_accuracy: 0.9670 - val_loss: 0.2370
 Epoch 79/100
 182/182 1s 3ms/step -
 accuracy: 1.0000 - loss: 4.6779e-07 - val_accuracy: 0.9670 - val_loss: 0.2421
 Epoch 80/100
 182/182 1s 4ms/step -

accuracy: 1.0000 - loss: 4.3655e-07 - val_accuracy: 0.9670 - val_loss: 0.2381
 Epoch 81/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 3.6735e-07 - val_accuracy: 0.9670 - val_loss: 0.2470
 Epoch 82/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 3.8048e-07 - val_accuracy: 0.9560 - val_loss: 0.2492
 Epoch 83/100
 182/182 1s 5ms/step -
 accuracy: 1.0000 - loss: 3.5095e-07 - val_accuracy: 0.9670 - val_loss: 0.2518
 Epoch 84/100
 182/182 1s 5ms/step -
 accuracy: 1.0000 - loss: 2.8144e-07 - val_accuracy: 0.9670 - val_loss: 0.2536
 Epoch 85/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 2.5700e-07 - val_accuracy: 0.9670 - val_loss: 0.2558
 Epoch 86/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 2.2375e-07 - val_accuracy: 0.9670 - val_loss: 0.2545
 Epoch 87/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 2.0454e-07 - val_accuracy: 0.9670 - val_loss: 0.2626
 Epoch 88/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 1.8806e-07 - val_accuracy: 0.9670 - val_loss: 0.2578
 Epoch 89/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 1.6905e-07 - val_accuracy: 0.9670 - val_loss: 0.2597
 Epoch 90/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 1.5850e-07 - val_accuracy: 0.9670 - val_loss: 0.2634
 Epoch 91/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 1.3683e-07 - val_accuracy: 0.9670 - val_loss: 0.2692
 Epoch 92/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 1.2103e-07 - val_accuracy: 0.9670 - val_loss: 0.2739
 Epoch 93/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 1.1226e-07 - val_accuracy: 0.9670 - val_loss: 0.2728
 Epoch 94/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 1.0208e-07 - val_accuracy: 0.9670 - val_loss: 0.2720
 Epoch 95/100
 182/182 1s 4ms/step -
 accuracy: 1.0000 - loss: 8.6491e-08 - val_accuracy: 0.9560 - val_loss: 0.2788
 Epoch 96/100
 182/182 1s 4ms/step -

```

accuracy: 1.0000 - loss: 8.4669e-08 - val_accuracy: 0.9670 - val_loss: 0.2822
Epoch 97/100
182/182          1s 4ms/step -
accuracy: 1.0000 - loss: 8.5563e-08 - val_accuracy: 0.9670 - val_loss: 0.2780
Epoch 98/100
182/182          1s 4ms/step -
accuracy: 1.0000 - loss: 7.2984e-08 - val_accuracy: 0.9670 - val_loss: 0.2912
Epoch 99/100
182/182          1s 4ms/step -
accuracy: 1.0000 - loss: 6.6209e-08 - val_accuracy: 0.9670 - val_loss: 0.2835
Epoch 100/100
182/182          1s 4ms/step -
accuracy: 1.0000 - loss: 5.5405e-08 - val_accuracy: 0.9670 - val_loss: 0.2840

```

```
[35]: print(history.history)
```

```

{'accuracy': [0.9313187003135681, 0.9862637519836426, 0.9835164546966553,
0.9890109896659851, 0.9945054650306702, 0.9945054650306702, 0.9945054650306702,
0.9945054650306702, 0.9945054650306702, 0.9972527623176575, 1.0, 1.0, 1.0, 1.0,
1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0,
1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0,
1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0,
1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0,
1.0, 1.0, 1.0, 1.0, 1.0, 1.0], 'loss': [0.18131770193576813,
0.06632174551486969, 0.04665875807404518, 0.03527999669313431,
0.0280336681753397, 0.020093752071261406, 0.01677803508937359,
0.015534481033682823, 0.00999732967466116, 0.00755199883133173,
0.005809265188872814, 0.005428066477179527, 0.002842236775904894,
0.0029312388505786657, 0.0016707920003682375, 0.001480839098803699,
0.001314258435741067, 0.0011998137924820185, 0.0007314879330806434,
0.000582462118472904, 0.0004897672333754599, 0.0004011947603430599,
0.0003748027666006237, 0.00029386961250565946, 0.00026521761901676655,
0.00021500336879398674, 0.00018576315778773278, 0.00015889824135228992,
0.00013918349577579647, 0.000122842364362441, 0.00010065011883853003,
9.550756658427417e-05, 8.226433419622481e-05, 6.677911733277142e-05,
7.461448694812134e-05, 5.2661984227597713e-05, 4.9428992497269064e-05,
4.1066945414058864e-05, 3.9625520003028214e-05, 3.558527896529995e-05,
2.9962233384139836e-05, 2.6870186047744937e-05, 2.235577085230034e-05,
2.124326056218706e-05, 2.0071302060387097e-05, 1.613554559298791e-05,
1.5775589417899027e-05, 1.2920587323606014e-05, 1.2320208043092862e-05,
1.024967150442535e-05, 9.594796210876666e-06, 8.749593689572066e-06,
7.734283826721366e-06, 6.943115749891149e-06, 6.372925781761296e-06,
5.500970019056695e-06, 5.0781609388650395e-06, 4.4483681449492e-06,
4.258748504071264e-06, 3.613254193624016e-06, 3.1357528769149212e-06,
2.7743690225179307e-06, 2.55941290561168e-06, 2.2186020487424685e-06,
2.3506725028710207e-06, 2.3254126517713303e-06, 1.6412931245213258e-06,
1.4794319440625259e-06, 1.31158878957649e-06, 1.2617134643733152e-06,

```

[illegible]

```

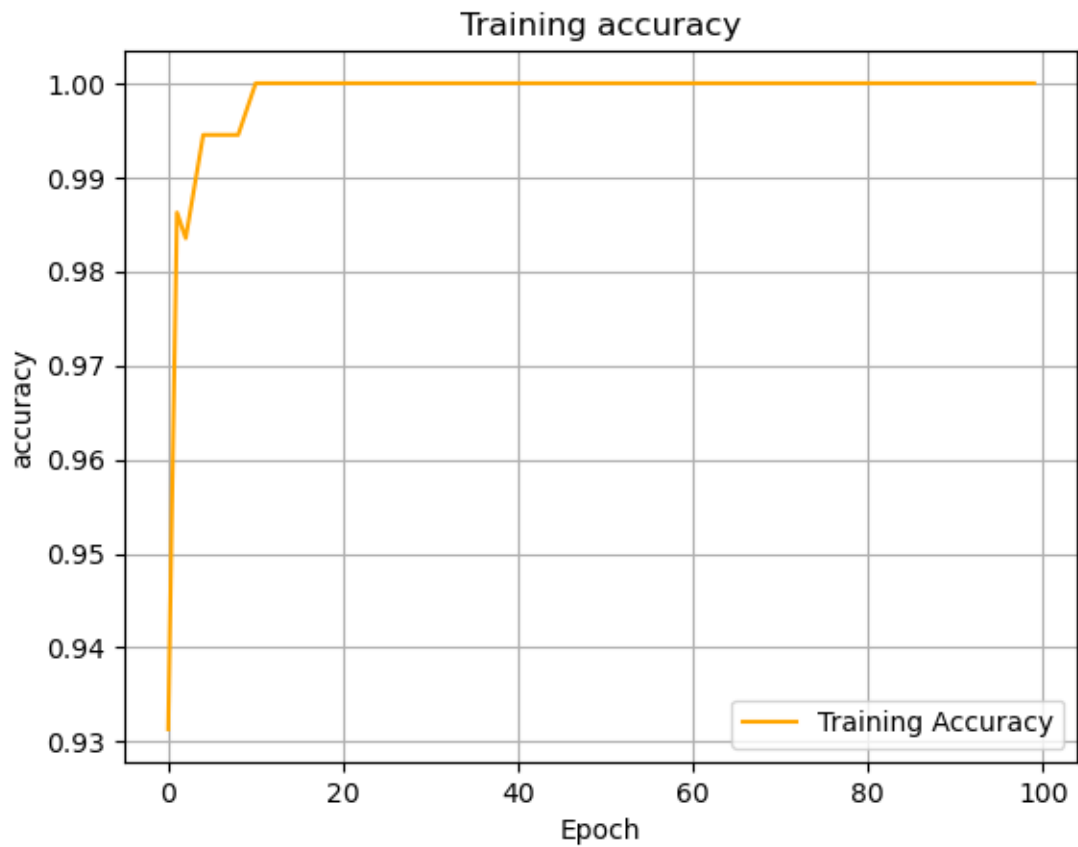
0.16716516017913818, 0.16765302419662476, 0.16697633266448975,
0.16693159937858582, 0.1733381748199463, 0.17112797498703003,
0.18284206092357635, 0.18533530831336975, 0.18701516091823578,
0.19026915729045868, 0.18438884615898132, 0.18310289084911346,
0.1870548129081726, 0.190188467502594, 0.19755756855010986, 0.19275403022766113,
0.2001558393239975, 0.20497684180736542, 0.20666205883026123,
0.20900064706802368, 0.20502954721450806, 0.20608586072921753,
0.21129697561264038, 0.20361082255840302, 0.2114747017621994,
0.21144144237041473, 0.22204269468784332, 0.21979549527168274,
0.220514178276062, 0.2253701090812683, 0.2214609980583191, 0.2299889326095581,
0.2301318198442459, 0.22620588541030884, 0.23092155158519745,
0.23989996314048767, 0.2413492500782013, 0.2322867512702942,
0.23699615895748138, 0.24214960634708405, 0.23811349272727966,
0.2470005303621292, 0.24920009076595306, 0.2517762780189514, 0.2536146342754364,
0.2557944655418396, 0.25446832180023193, 0.2626398205757141,
0.25775840878486633, 0.25965356826782227, 0.2633799612522125,
0.2691704034805298, 0.27385035157203674, 0.27280882000923157,
0.27199068665504456, 0.27875006198883057, 0.2821667492389679,
0.2779881954193115, 0.2912006080150604, 0.2835317552089691,
0.28400057554244995]}

```

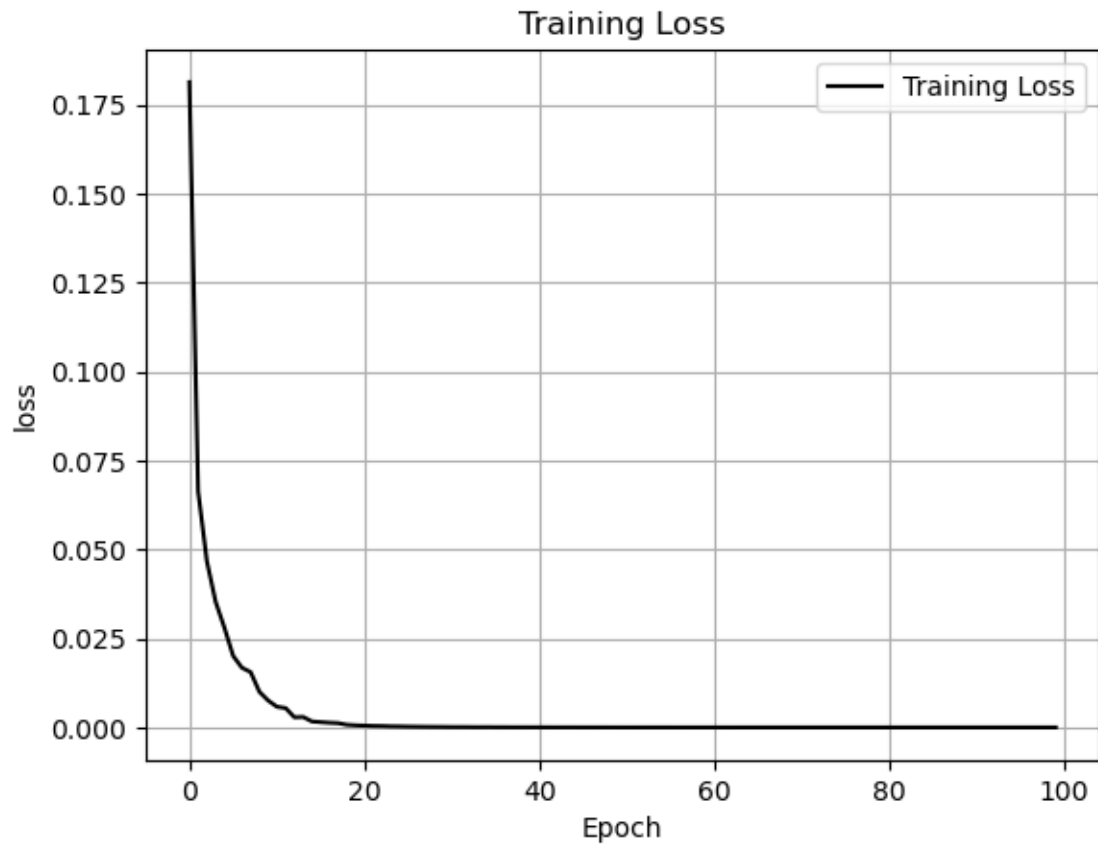
```

[46]: plt.plot(history.history["accuracy"], label = "Training Accuracy",color = "orange")
plt.xlabel("Epoch")
plt.ylabel("accuracy")
plt.title("Training accuracy")
plt.legend()
plt.grid(True)
plt.show()

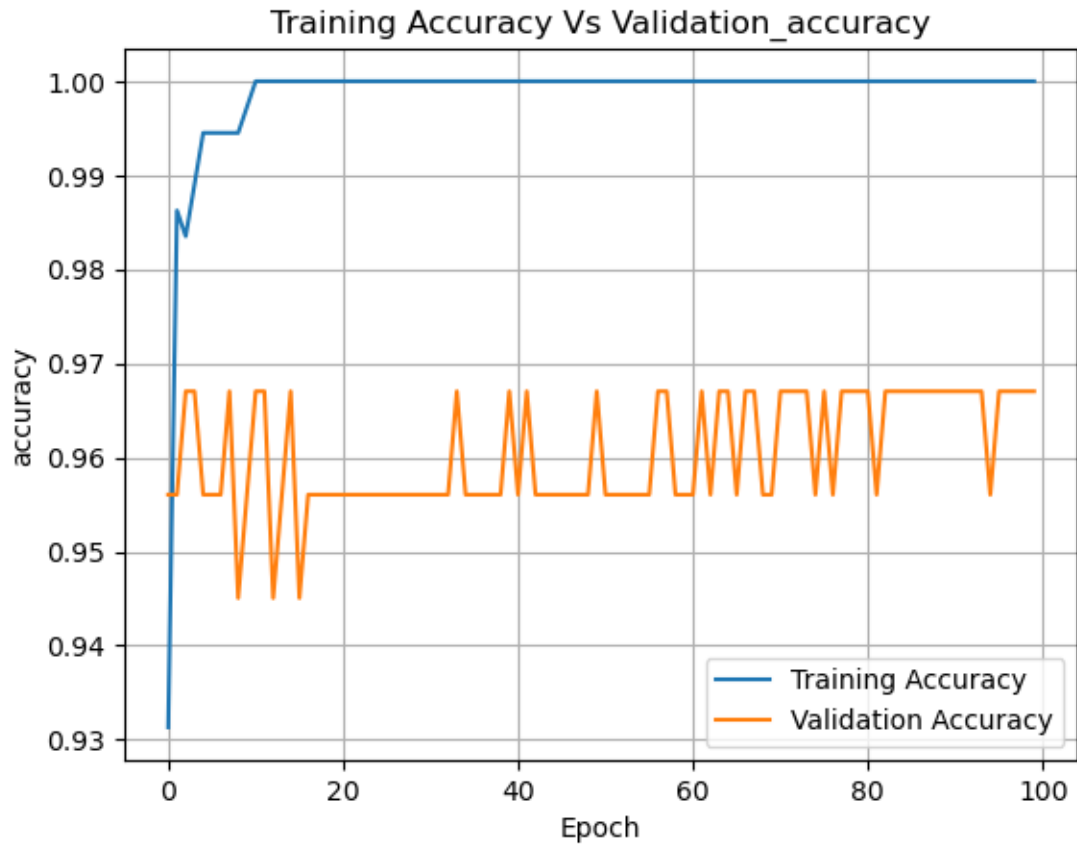
```



```
[48]: plt.plot(history.history["loss"], label = "Training Loss", color = "black")
plt.xlabel("Epoch")
plt.ylabel("loss")
plt.title("Training Loss")
plt.legend()
plt.grid(True)
plt.show()
```



```
[38]: plt.plot(history.history["accuracy"], label = "Training Accuracy")
plt.plot(history.history["val_accuracy"], label = "Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("accuracy")
plt.title("Training Accuracy Vs Validation_accuracy")
plt.legend()
plt.grid(True)
plt.show()
```



```
[39]: plt.plot(history.history["loss"],label = "Training Loss")
plt.plot(history.history["val_loss"],label = "Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("loss")
plt.title("Training Loss Vs Validation_loss")
plt.legend()
plt.grid(True)
plt.show()
```



```
[40]: print(history.history.keys())
```

```
dict_keys(['accuracy', 'loss', 'val_accuracy', 'val_loss'])
```

```
[41]: loss, accuracy = models.evaluate(X_trained_scaled, y_train)
      print(f"Loss : {loss}")
      print(f"accuracy :{accuracy}")
```

```
15/15          0s 6ms/step -
accuracy: 0.9934 - loss: 0.0568
Loss : 0.056800153106451035
accuracy :0.9934065937995911
```

```
[ ]:
```